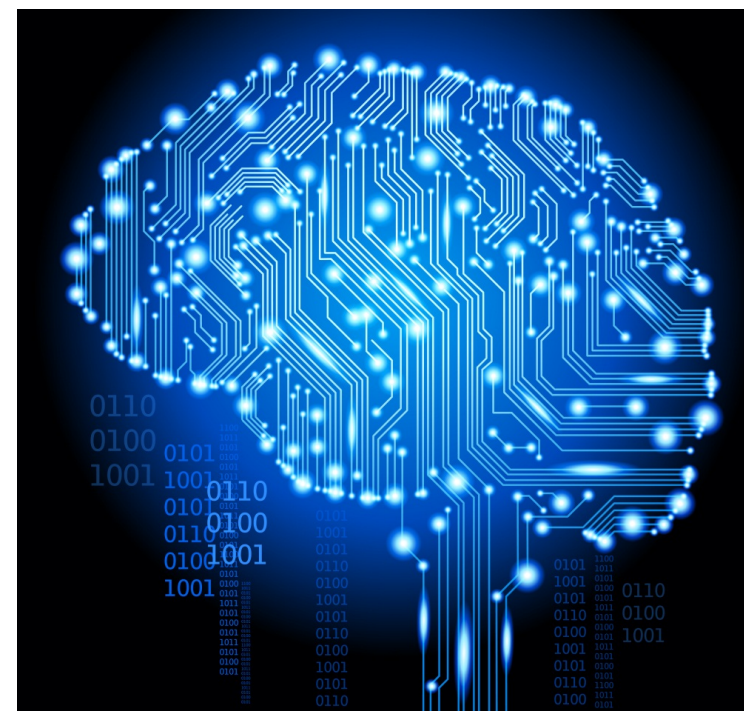


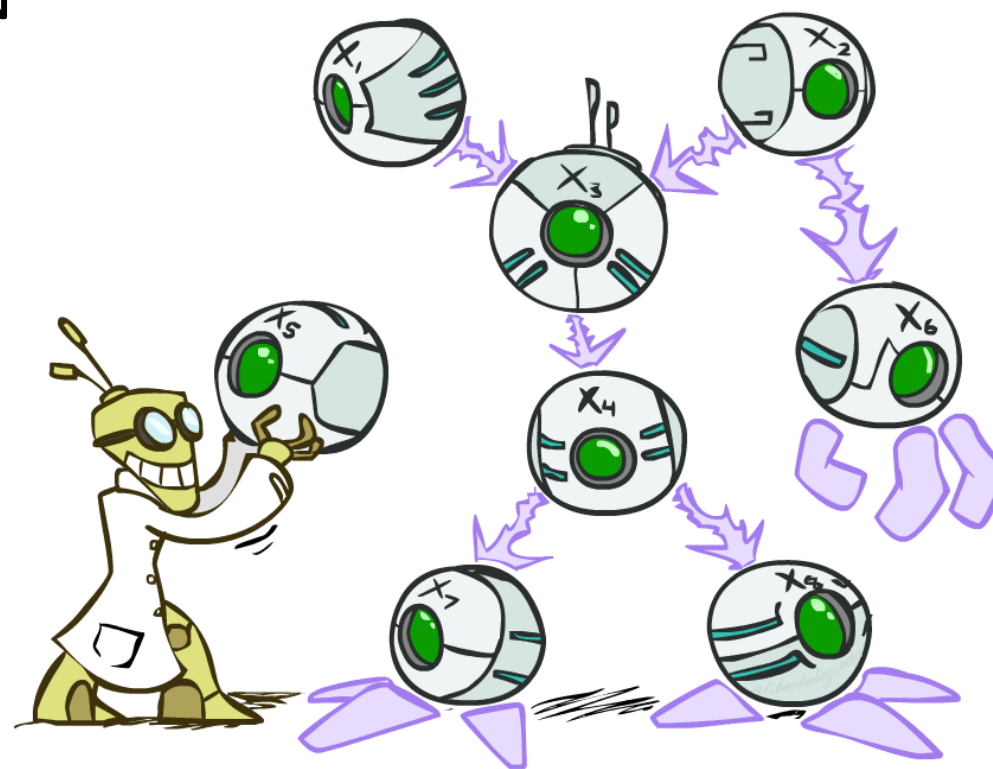
深度神经网络 II

Deep Neural Networks



内容

- 深度学习常用模型
 - Convolutional Neural Network, CNN
 - Recurrent Neural Networks, RNN
 - LSTM

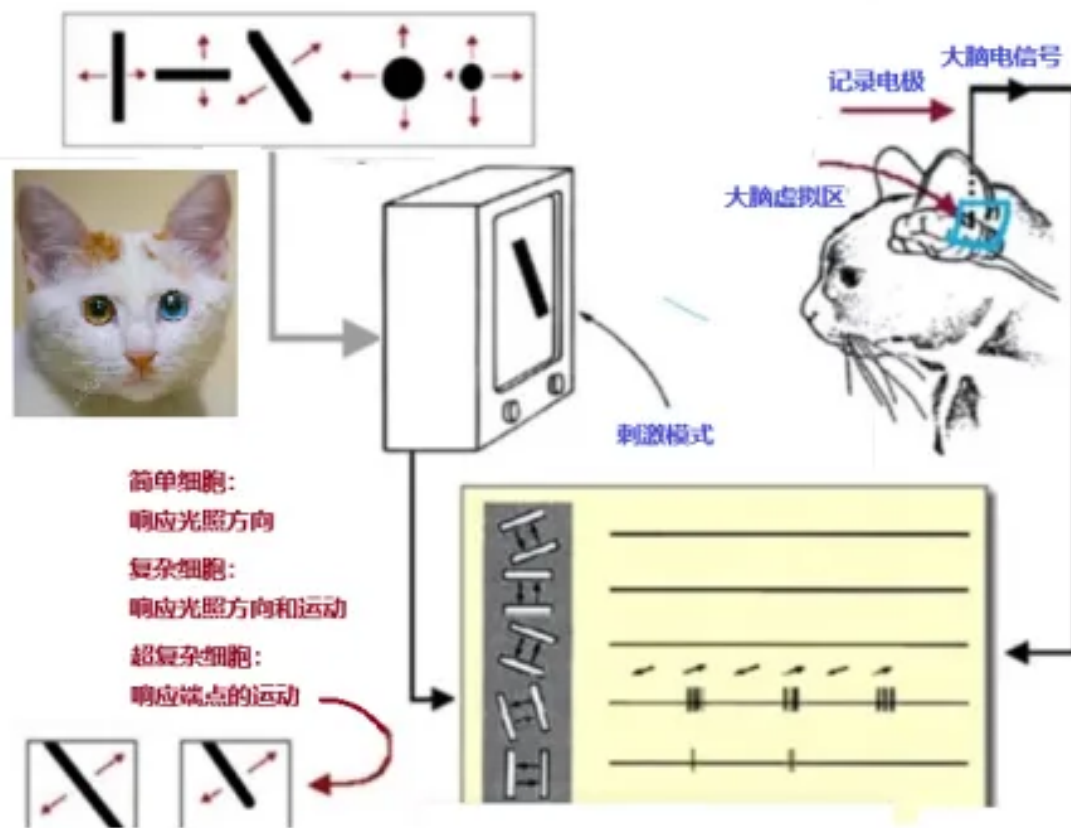
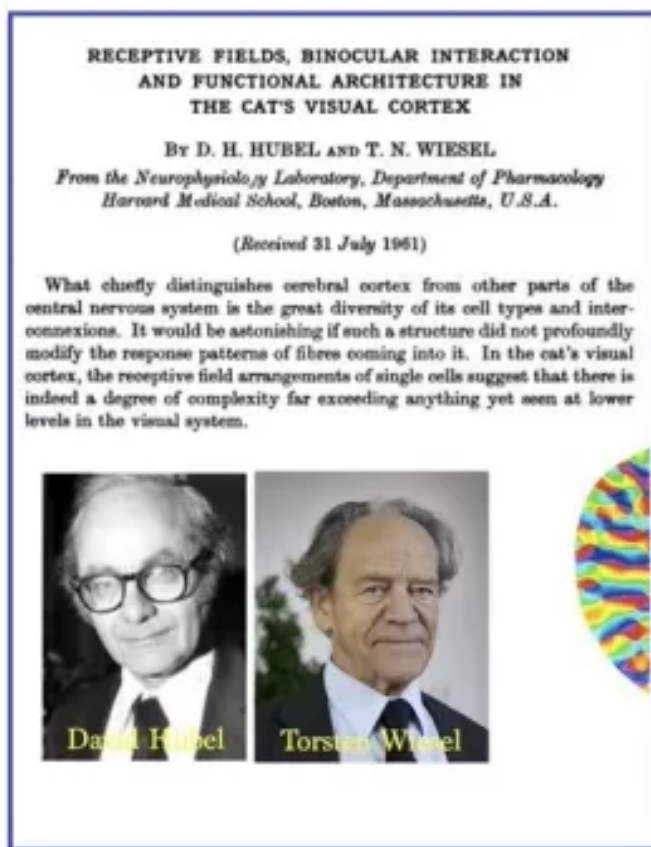


Deep Learning的常用模型

——Convolutional Neural Networks, CNN

CNN卷积神经网络-生物学动机

- 20世纪60年代，Hubel和Wiesel研究猫脑皮层
 - 研究视觉模式与特定神经元的关系



CNN卷积神经网络-生物学动机

■Hubel和Wiesel发现

- 猫的初级视觉皮层中单个神经元具有感受野，且感受野很小，初级视觉皮层是负责处理视觉刺激的局部特征信息
- 初级视觉皮层神经元还对刺激特征有选择性，如方向、空间频率选择性等。比如只有当刺激线条处于适当的角度时，细胞才表现出最大的兴奋。
- 猫的初级视皮层- Hubel-Wiesel结构
 - 简单细胞：只对特定的空间位置和方向具有强烈的反应
 - 复杂细胞：复杂细胞具有更大的接受域，其对于特征位置的微小偏移具有不变性

CNN卷积神经网络-生物学动机

- 基于此，构建了卷积神经网络特殊的深层神经网络结构
 - 它的神经元间的连接是非全连接的，具有局部性
 - 同一层中某些神经元之间的连接的权重是共享的（即相同的）降低了网络的复杂性
 - 两层神经网络模拟初级视皮层中的简单细胞和复杂细胞
 - 每层的神经元被组织成二维平面
 - “简单细胞”层提取其输入中的局部特征
 - “复杂细胞”层组合“简单细胞”层中相应的子区域，使得整个网络对局部变换具有一定的不变性。

全链接网络存在的问题

- 全连接神经网络存在的问题

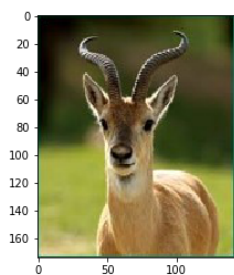
- 复杂度高
- 实际中，不同的像素排列顺序代表不同的图像
- 全连接网络，像素的输入顺序与网络的性能无关
 - 所有的输入像素都是平等关系
 - 相邻的输入元素与不相邻的输入元素同等对待
 - 但是实际中，不同的像素排列顺序代表不同的图像
- 网络无法使用图像的空间信息，即像素之间的空间联系



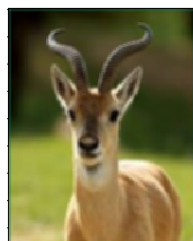
卷积的作用

■ 图像的空间联系具有局部性

- 局部的像素联系较为紧密，而距离较远的像素相关性则较弱
- 如何抽取图像的局部空间特征：卷积可用于图像增强与特征提取



1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9



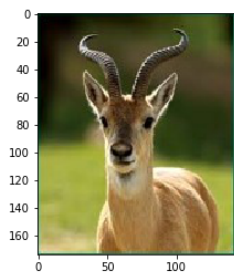
模糊



*

0	0.2	0
0.2	0.2	0.2
0	0.2	0

=



0	-1	0
-1	5	-1
0	-1	0



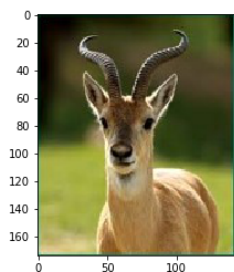
锐化



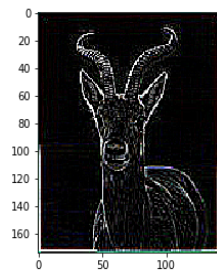
*

0	-1	0
-1	5	-1
0	-1	0

=



-1	-1	-1
-1	8	-1
-1	-1	-1

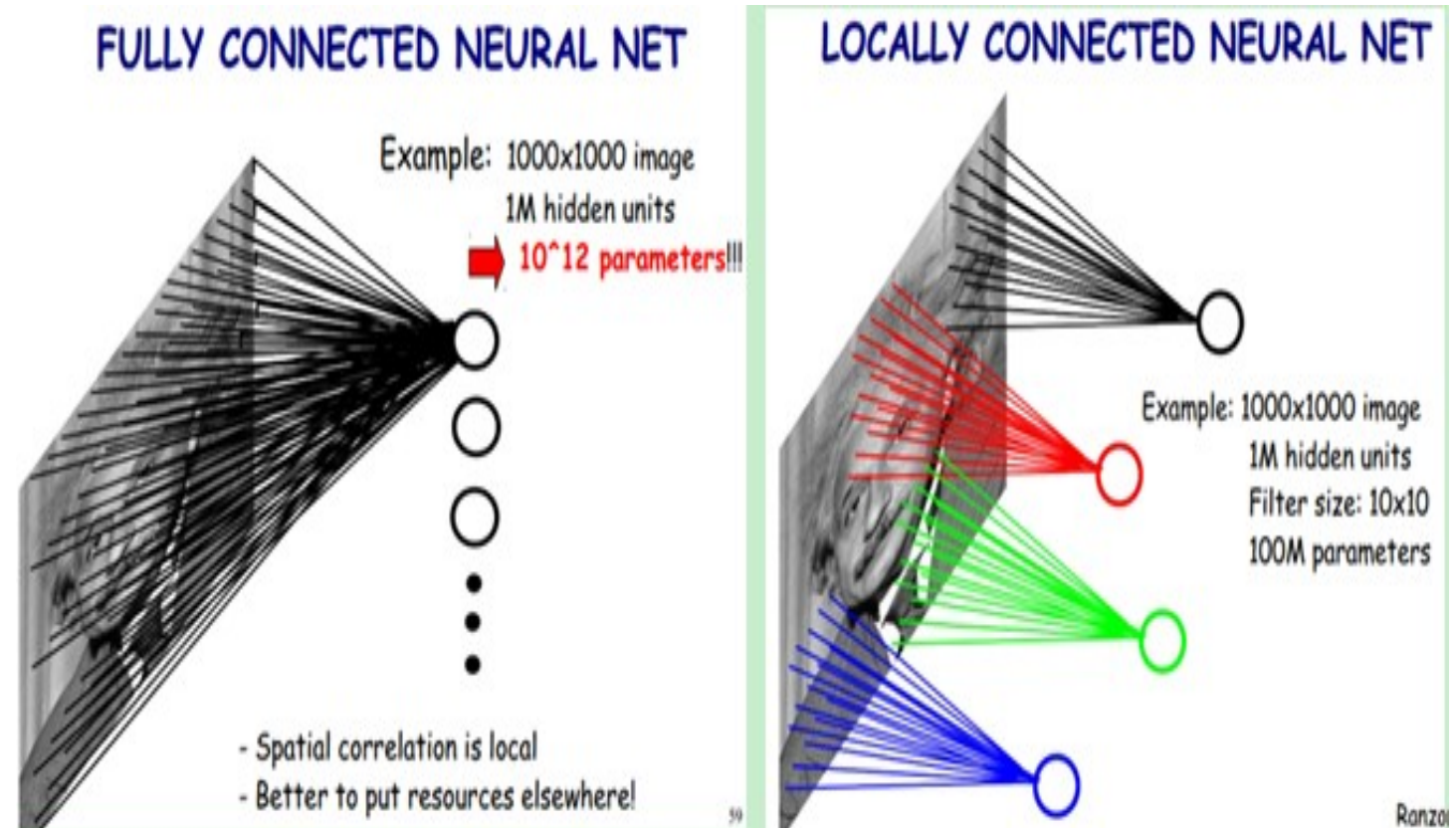
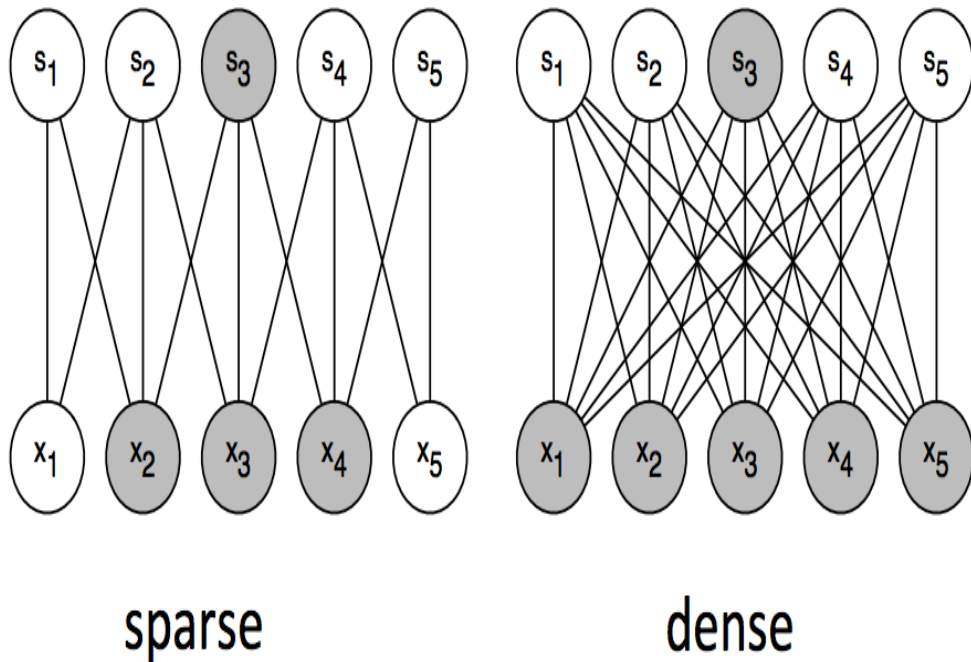


提取边缘

CNN局部链接

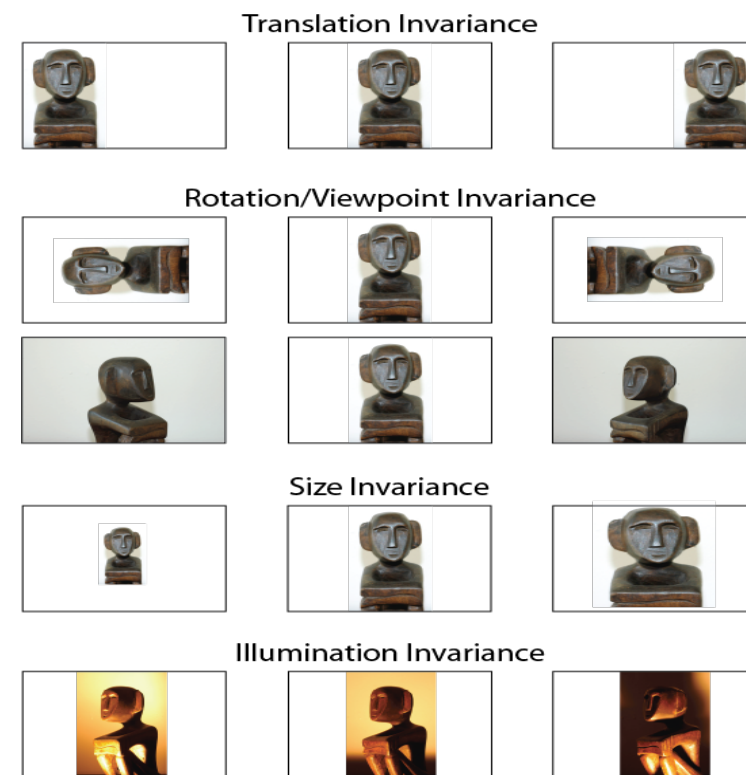
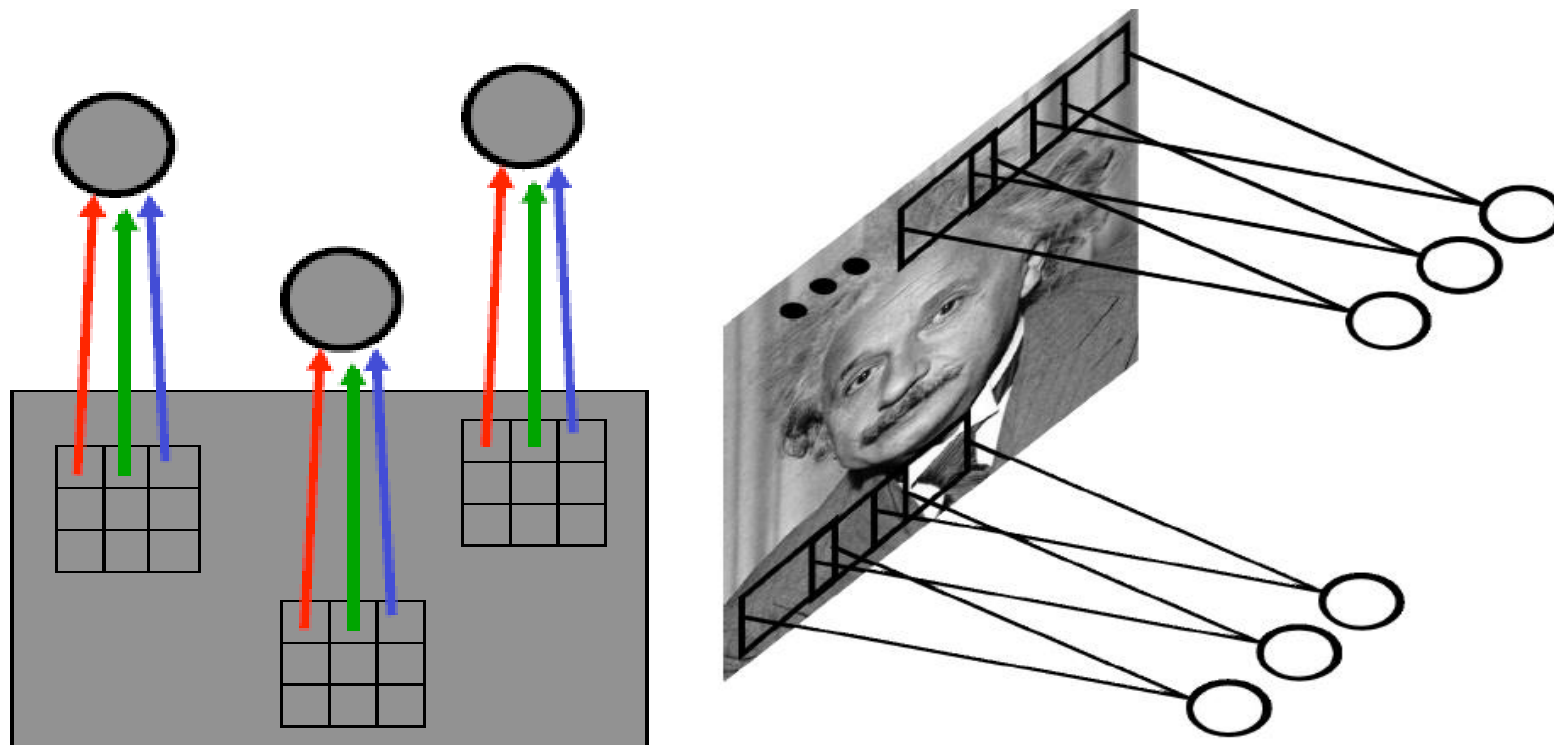
■ 局部感知野

- 图像的空间联系具有局部性：局部的像素联系较为紧密，而距离较远的像素相关性则较弱。
- 神经元的局部链接便于更好学习图像的局部特征，同时减少了需要训练的权值数目，(网络链接由稠密链接变为稀疏链接)



CNN-局部连接

- 参数共享：不同神经元共享权值，实现参数共享
 - 共享权值的神经元功能相同，可以抽取相同的局部特征
 - 增加神经元的数目，使其覆盖全部输入，可在输入的不同位置检测同一种特征
 - CNN具有平移不变性(translation invariance)



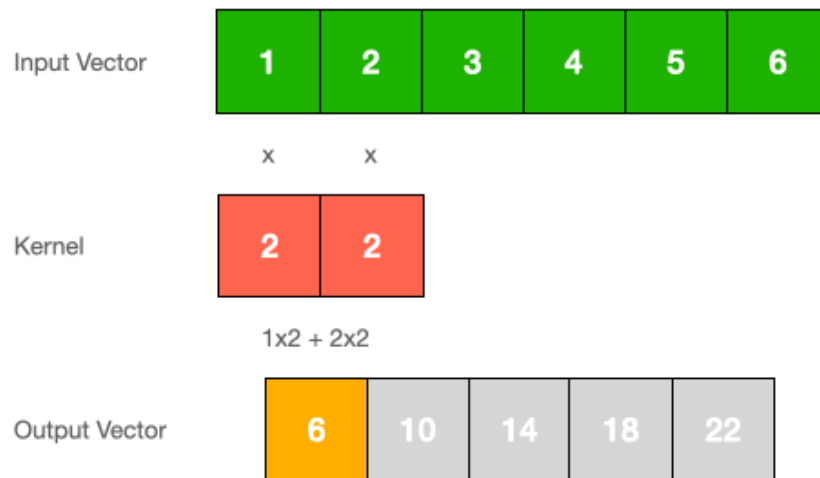
卷积(Convolution)与卷积核(kernel,Filter)

- 一维卷积:输入一维，卷积核一维

$$C_j = \mathbf{m}^T \mathbf{x}_{j-|\mathbf{m}|+1:j}$$

Kernel size:1x2

卷积后的输出也称为Feature Map



Kernel 2

```
nn.Conv1d(in_channels=1, out_channels=1,  
          kernel_size=2,  
          bias=False  
)
```

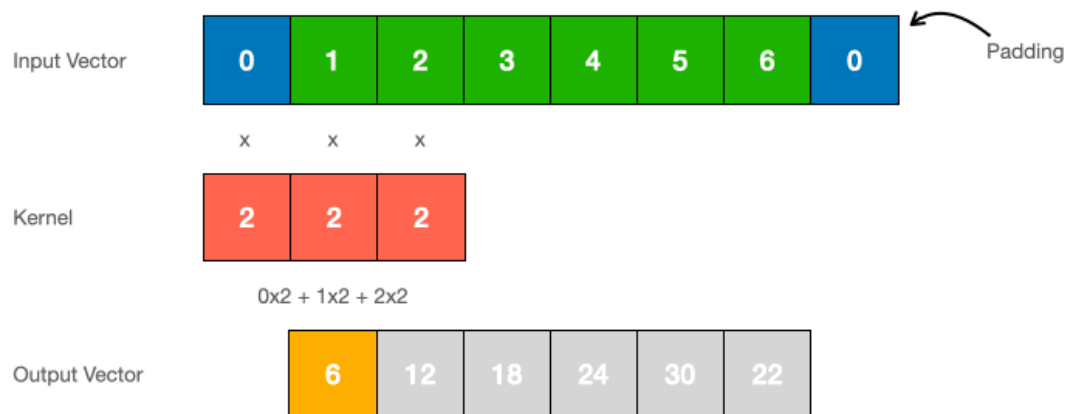
Size

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$
$$5 = \frac{(6 + (2 * 0) - 1 * (2 - 1) - 1)}{1} + 1$$

卷积(Convolution)与卷积核(kernel,Filter)

■ 一维卷积:padding

核越大，输出向量就越小。有时，我们希望生成与输入向量长度相同的特征向量。我们可以在输入向量的开头和结尾添加零。



Padding

```
nn.Conv1d(in_channels=1, out_channels=1,  
          kernel_size=3, padding=1,  
          bias=False  
)
```

Size

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$
$$6 = \frac{6 + (2 * 1) - 1 * (3 - 1) - 1}{1} + 1$$

卷积(Convolution)与卷积核(kernel,Filter)

13

- **Padding:** valid and same(左奇右偶)

- "VALID" = without padding:

inputs: 1 2 3 4 5 6 7 8 9 10 11 (12 13)
 |-----|
 |-----|
 |-----|

- "SAME" = with zero padding:

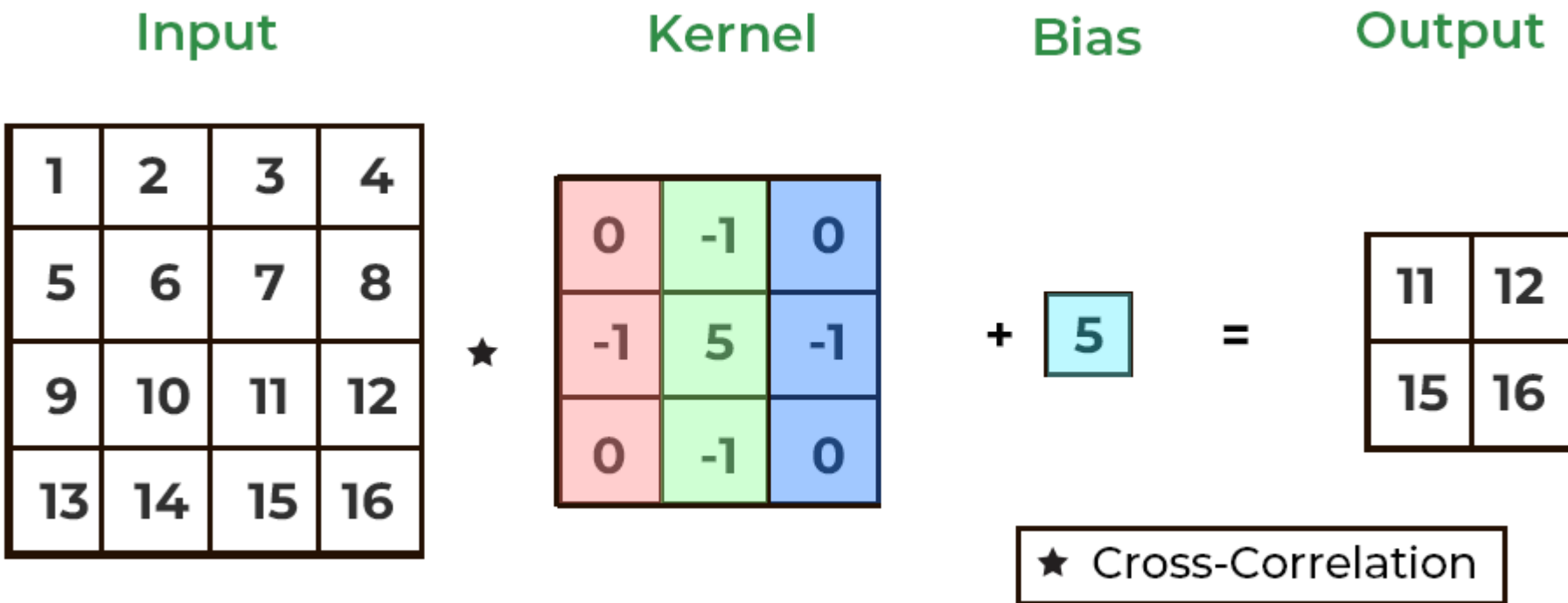
[illegible]

this example:

- Input width = 13
- Filter width = 6
- Stride = 5

卷积(Convolution)与卷积核(kernel,Filter)

- 二维卷积:输入二维，卷积核二维



卷积(Convolution)与卷积核(kernel, Filter)

15

二维卷积操作后的图像大小:

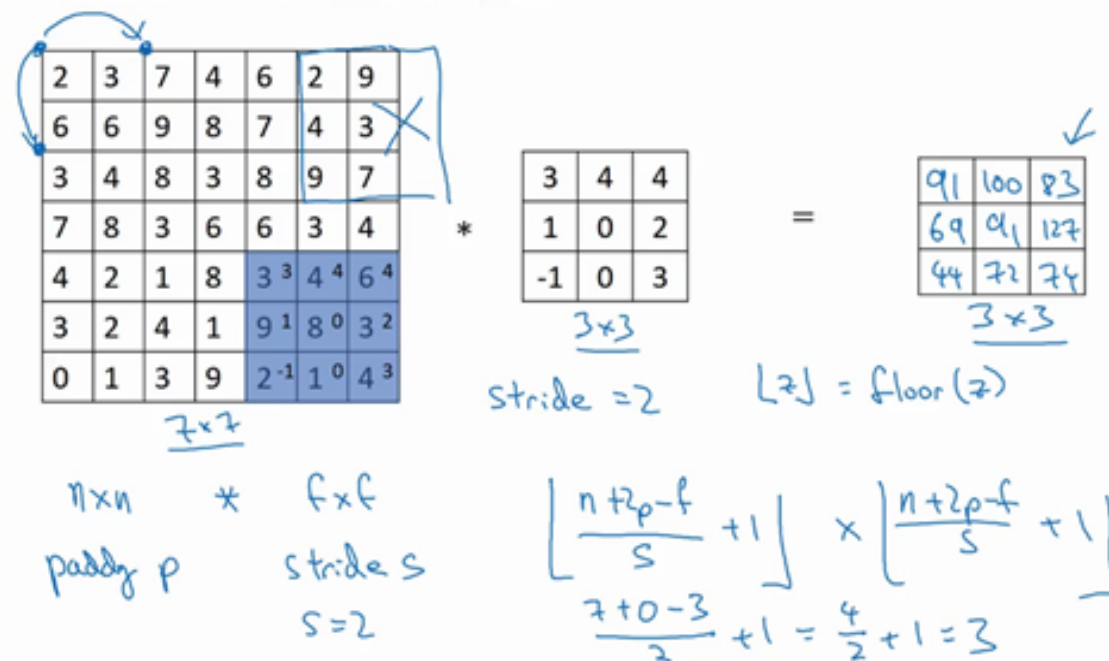
卷积步长 s
图像尺寸 W, H, n
Filter F , 或 f
Padding p

$$\text{output width} = \frac{W - F_w + 2P}{S_w} + 1$$

$$\text{output height} = \frac{H - F_h + 2P}{S_h} + 1$$

考虑了水平和垂直步长不同, 及卷积核维度不同

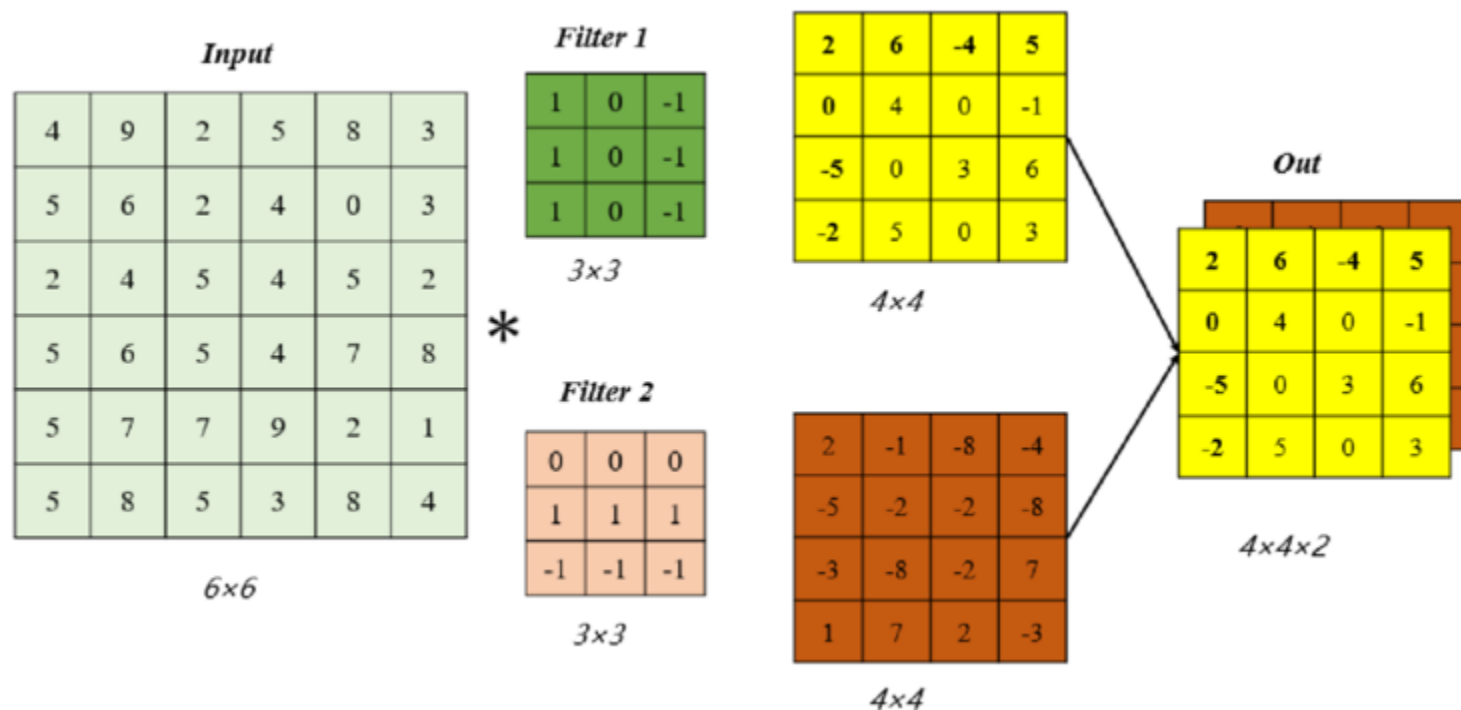
Strided convolution



$$n = 7, p = 0, f = 3, s = 2, \frac{7 + 0 - 3}{2} + 1 = 3$$

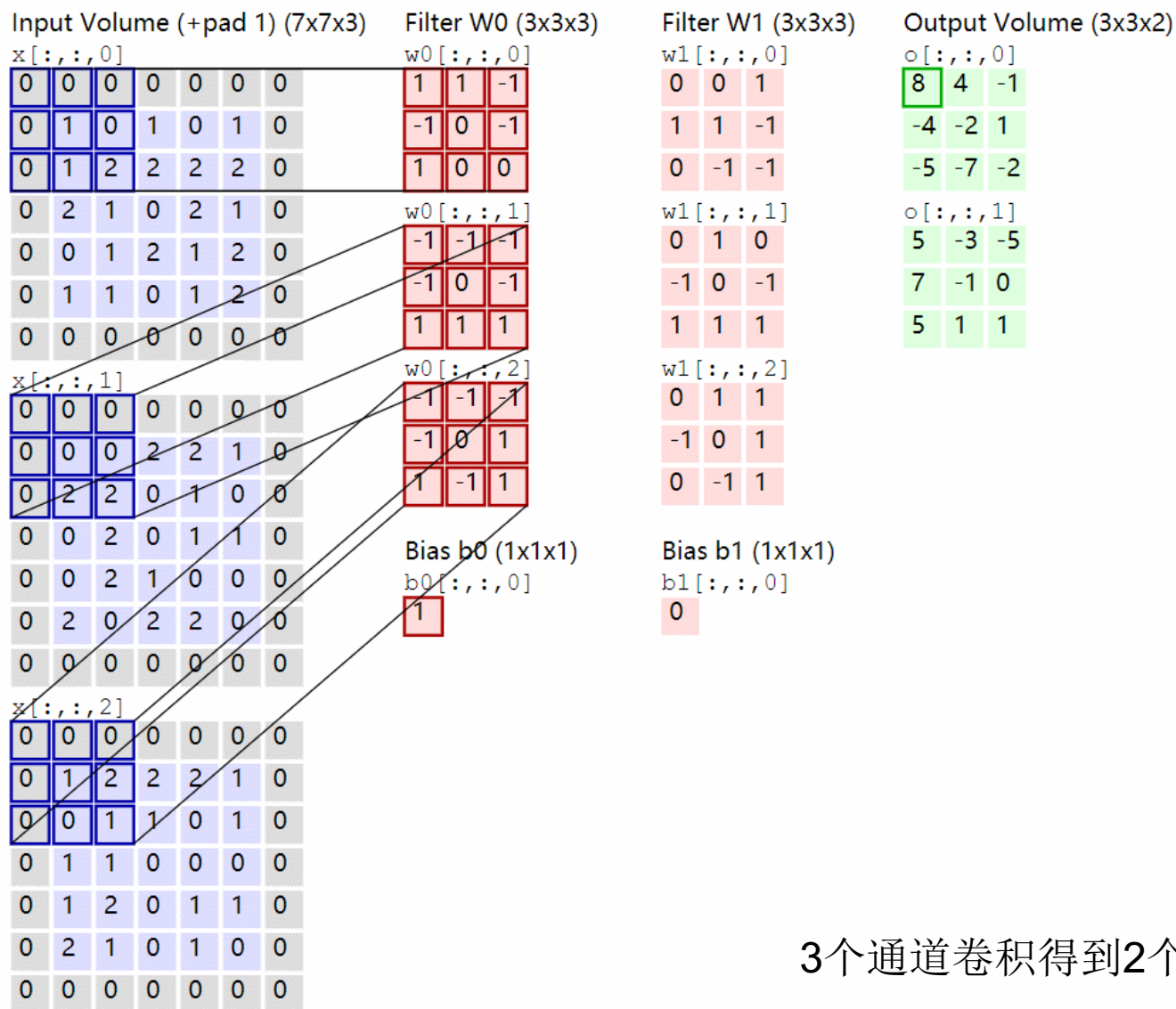
多卷积核

- 如果需要抽取图像的多种特征，可使用多个卷积核
 - 多个卷积核可以生成多幅特征图，它们可以看做是一张图像的不同特征表示
 - 每个特征图可以看作为一个通道(channel)

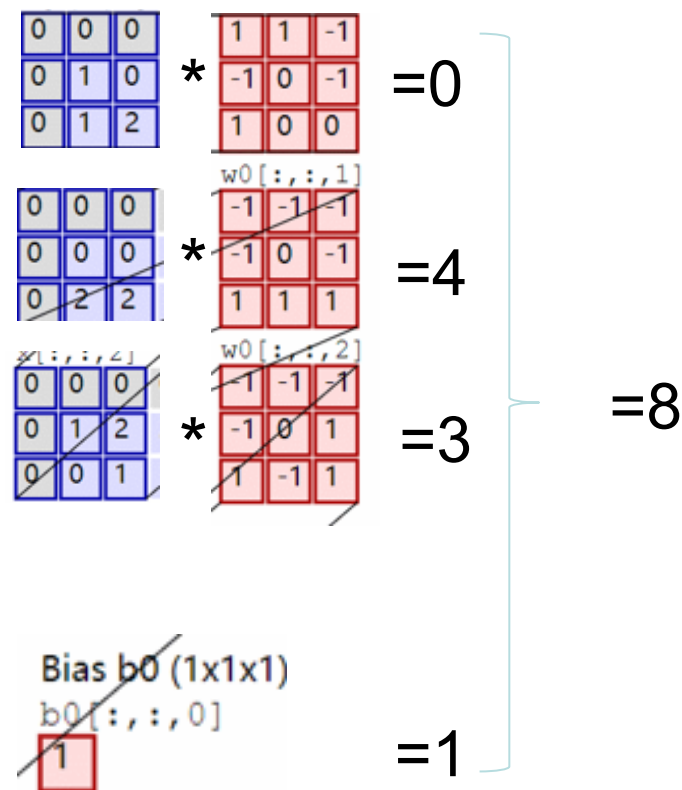


多通道卷积核与多通道卷积计算

17



$$8 = 0 + 4 + 3 + 1$$

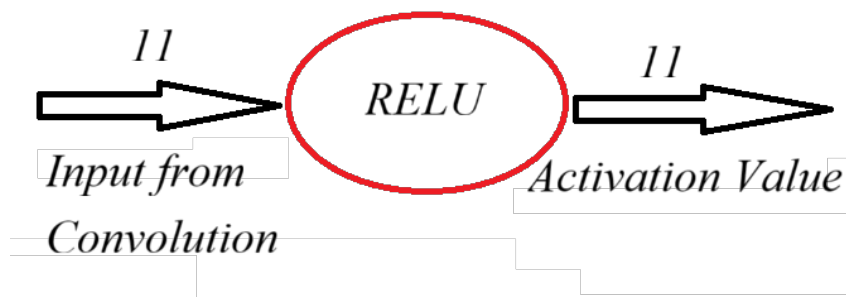
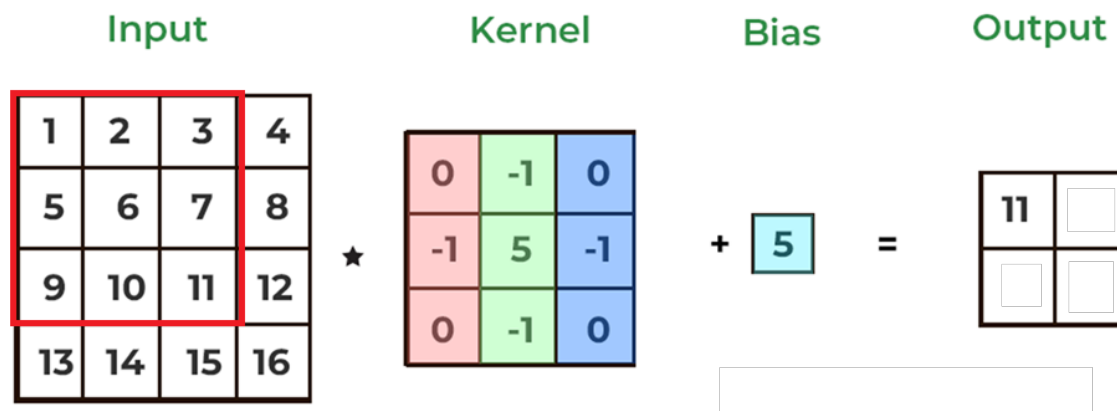
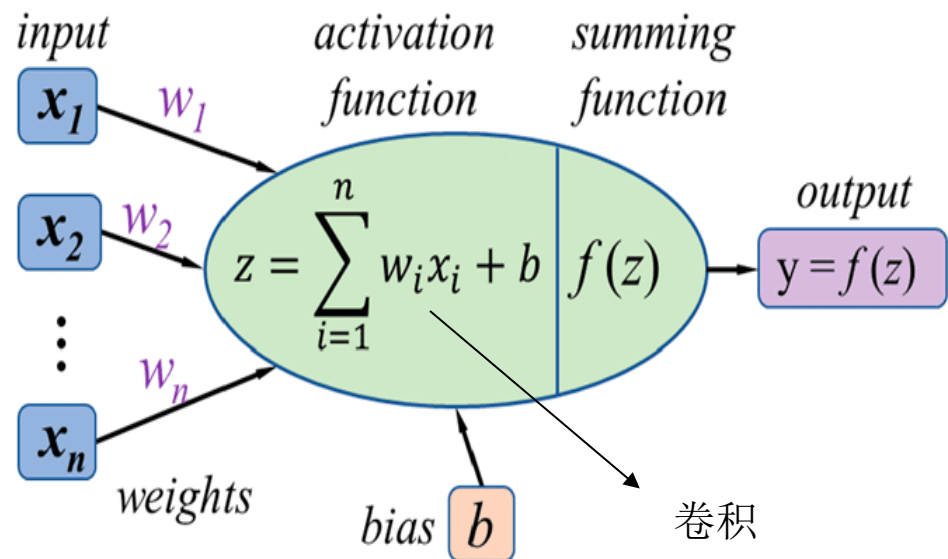


3个通道卷积得到2个通道的过程

卷积层神经元与卷积核关系

18

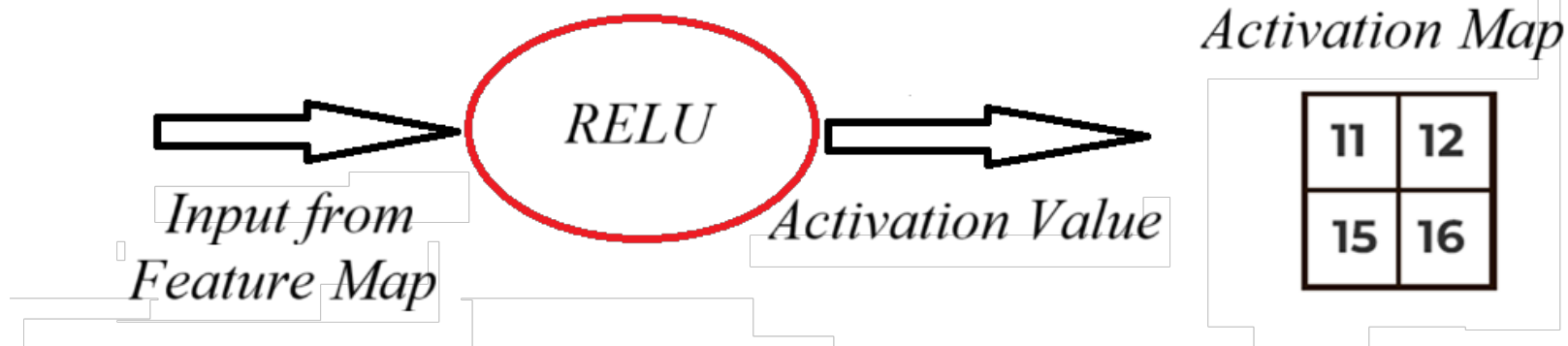
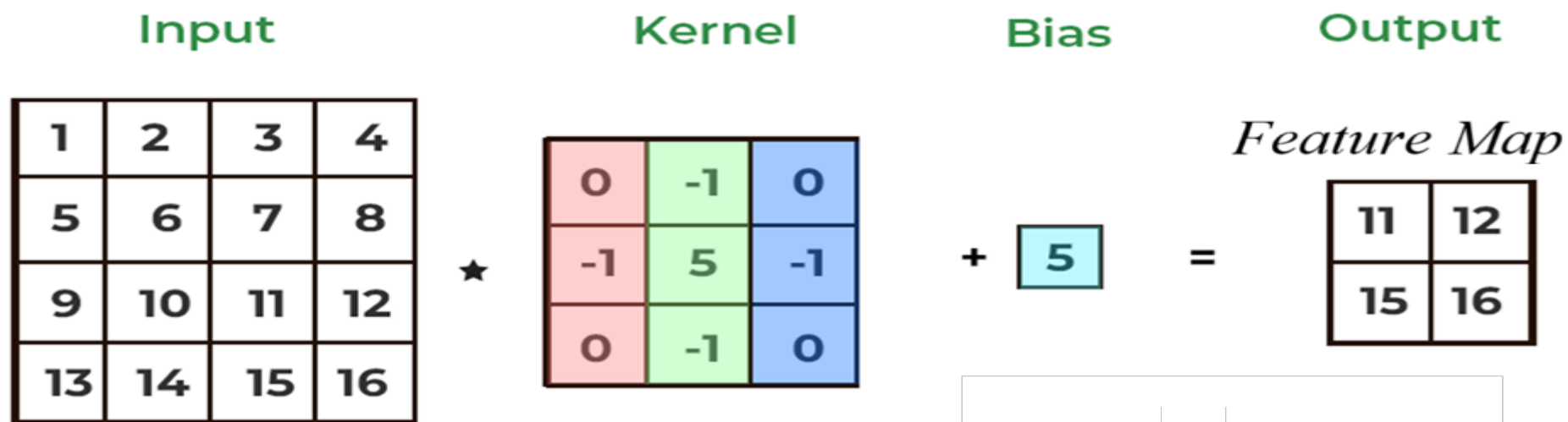
- 卷积核的每个数值对应卷积层神经网络的一个输入链接的权重
- 一个卷积计算的结果作为卷积层一个神经元的净输入，卷积层神经元使用Relu函数作为活跃函数
- 同一个卷积核对应的神经元有相同的bias，或者每个卷积核有一个bias
- 卷积层要学习的参数为卷积核及bias



卷积层神经元与Activation Map

19

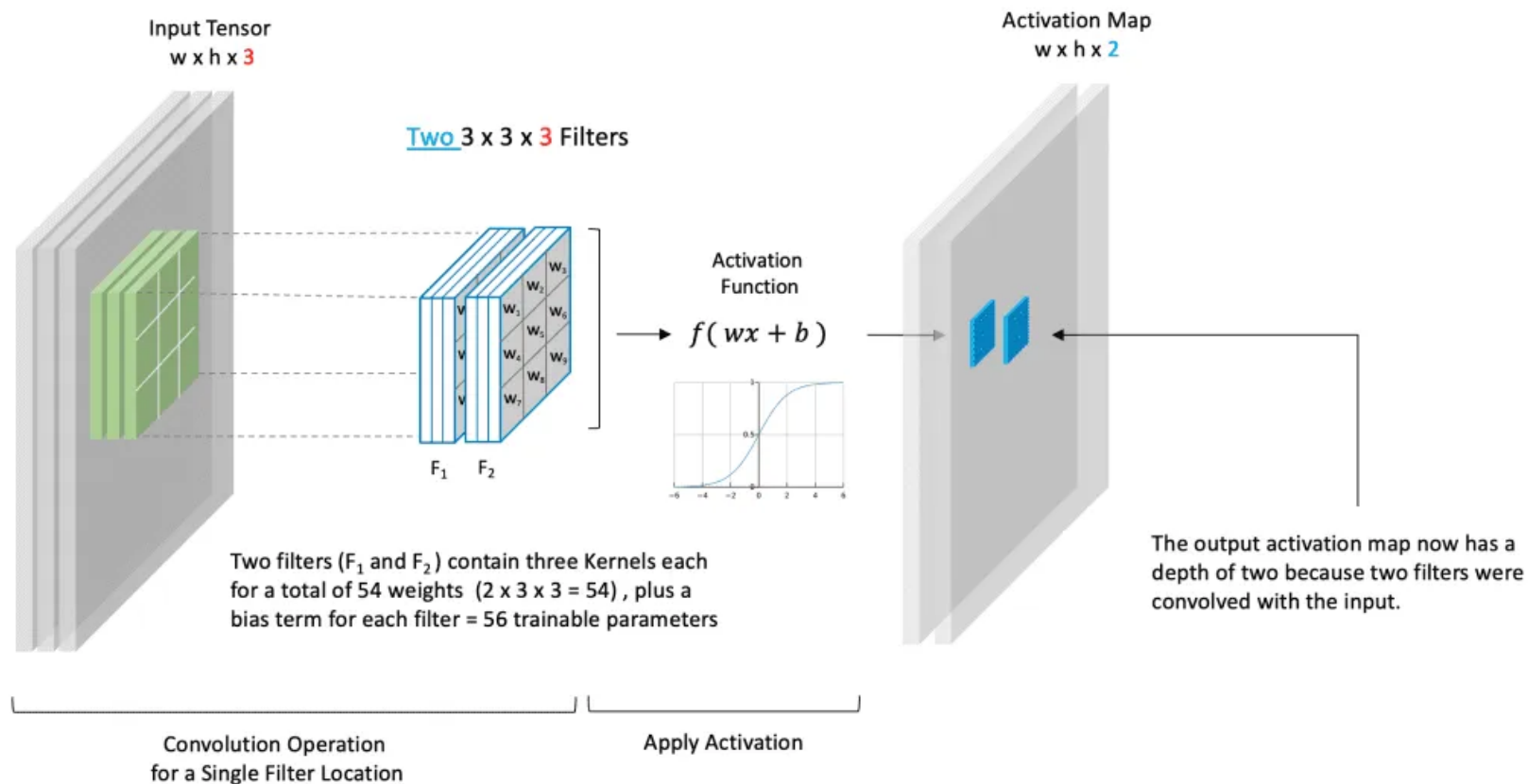
卷积计算的结果做为卷积层神经元的输入,所有神经元的输出做成Activation Map:



卷积层神经元与Activation Map

20

多通道卷积的Activation Map



池化层与池化(pooling)

■池化层对卷积层输出的**Activation Map**，进行池化操作

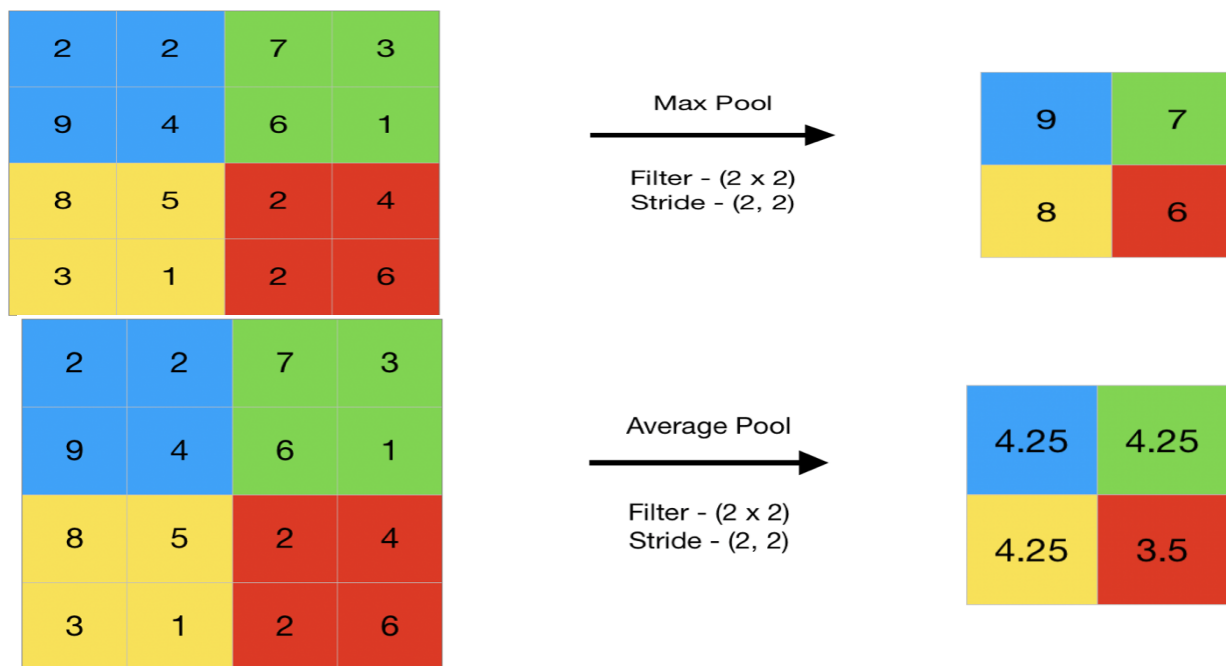
- Pooling操作与卷积相似, 可设置池化核的维度及步长, 对Activation Map不同位置的特征数据进行聚合统计
- 池化层无学习参数, 池化后的输出为Pooled Feature Map
- 池化运算(池化核函数)

- Max pool

- Average pool

$$\max\{x_i\}$$

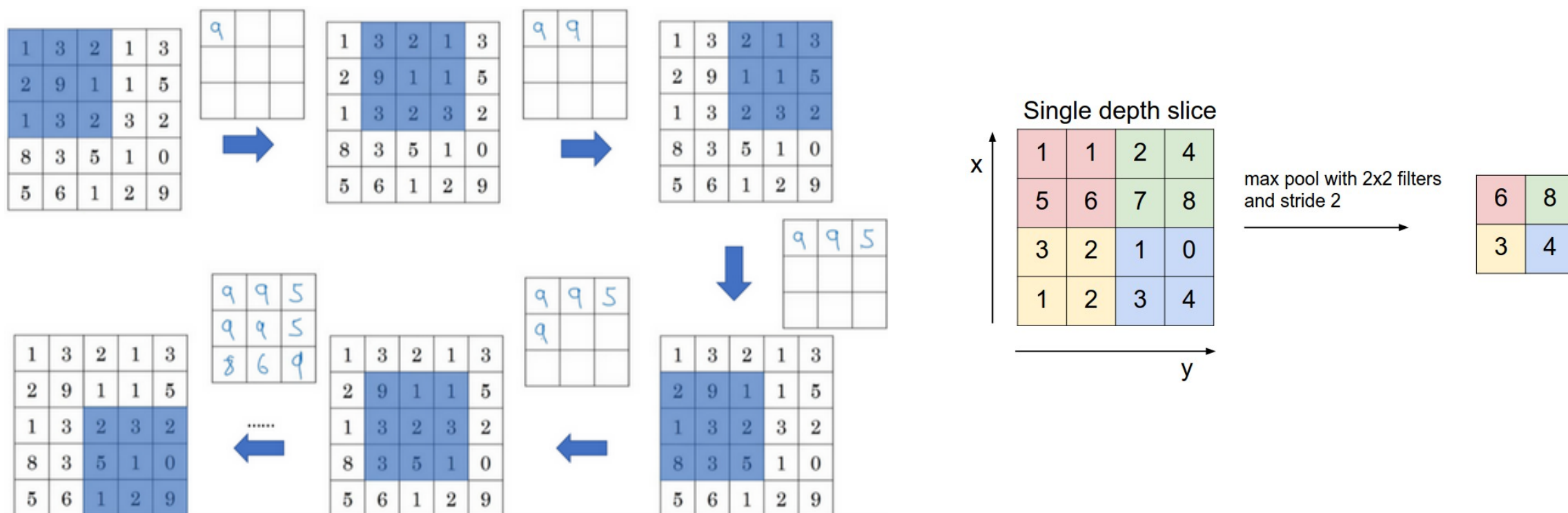
$$\frac{1}{s^2} \sum x_i$$



池化层与池化(pooling)

22

池化操作后的特征图维度: n^H 池化核高, n^W 池化核宽, n^c 通道数, s 移动步长



最大池化的输入就是 $n^H \times n^W \times n^c$, 假设没有padding, 则输出 $\lfloor \frac{n^H - f}{s} + 1 \rfloor \times \lfloor \frac{n^W - f}{s} + 1 \rfloor \times n^c$

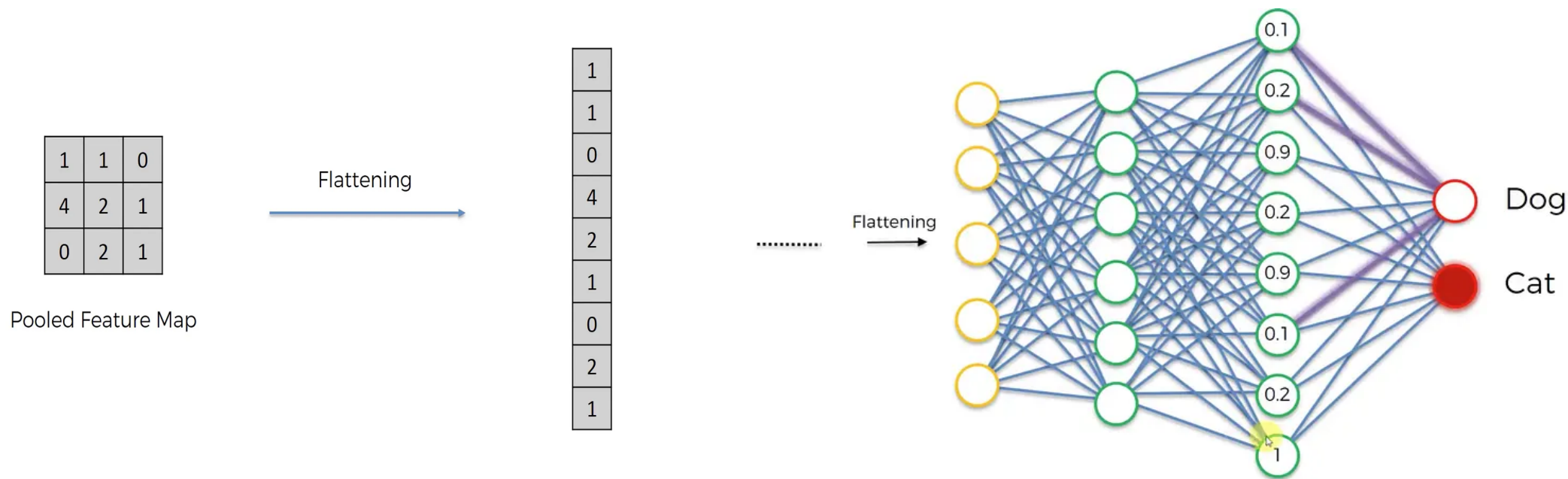
池化层与池化(pooling)

■ 池化的作用

- **降维**：池化层减少了特征图的空间大小，从而减少了网络中的参数和计算量。这使得模型更快、更高效。
- **平移不变性**：池化有助于网络对输入图像中的微小平移或扭曲保持不变。例如，即使图像中的对象略微移动，池化输出也将保持相对不变。
- **过度拟合预防**：通过减少空间维度，降低模型复杂性，防止过度拟合。
- **特征层次**：池化层有助于构建特征的层次表示，其中较低层捕获细节特征，较高层捕获更抽象和全局的特征。

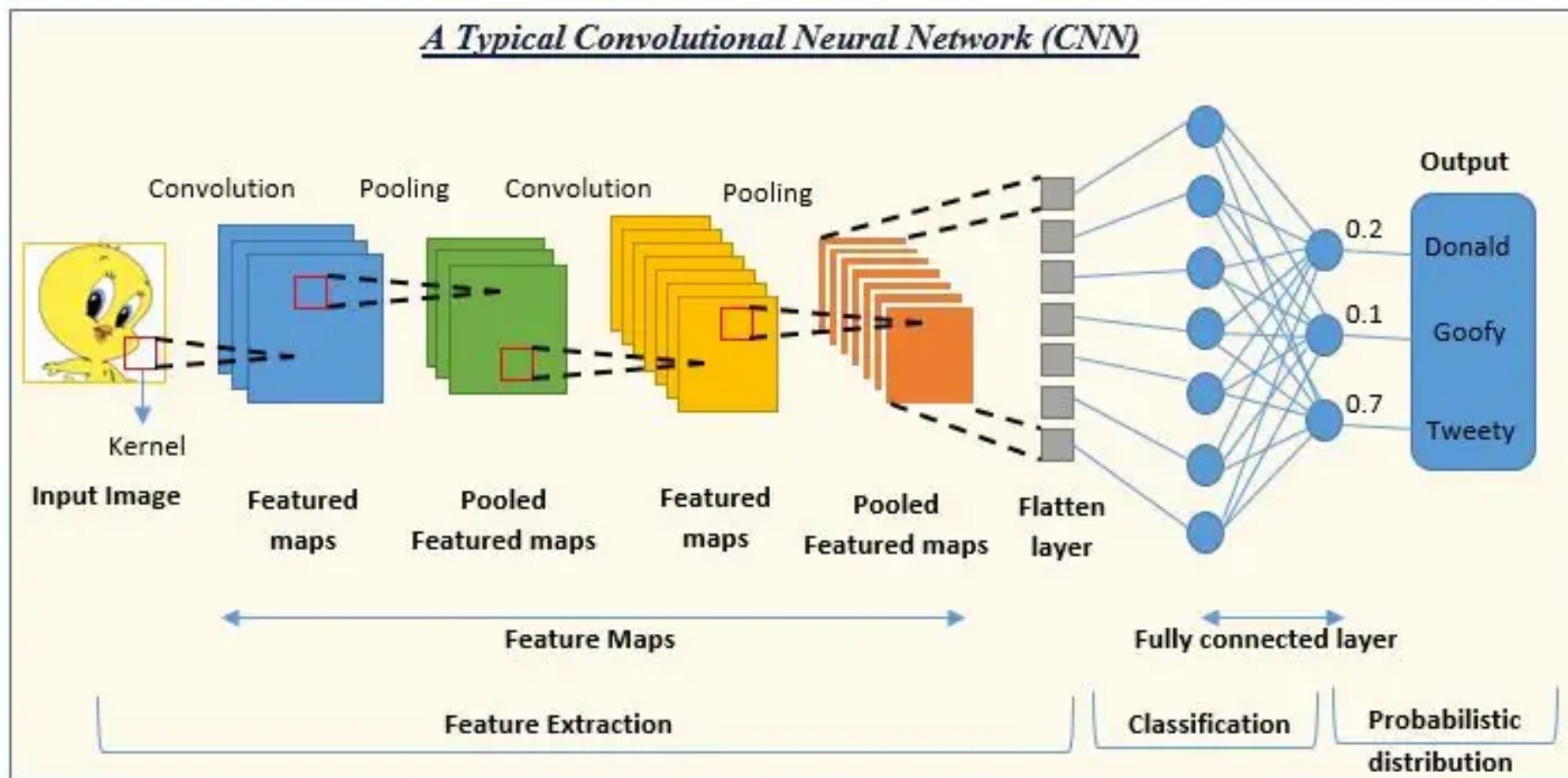
全连接层

- 获得了卷积或池化特征图之后，下一步利用这些特征去做分类
 - 将特征图平坦化之后作为全连接层的输入
 - 建立全连接层，全连接层可以为多层



CNN基本结构

- 卷积神经网络是一个多层的神经网络
 - 每层由平面组成
 - 每个平面由多个独立神经元组成（池化层除外）



CNN训练过程

- 监督训练

- Bp算法

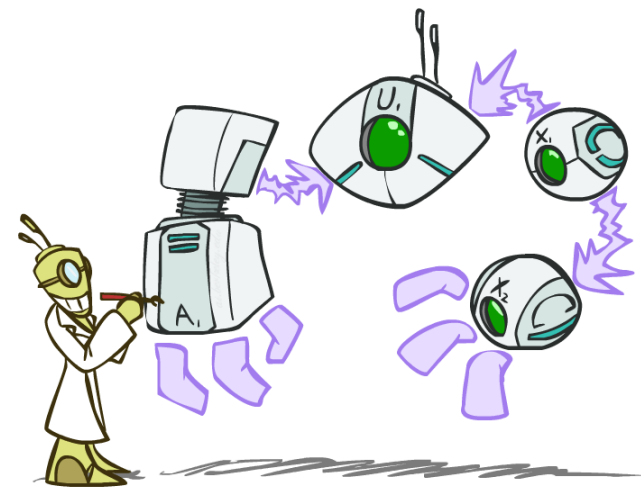
- 向前传播

从样本集中取一个样本 (X, Y_p) ，将 X 输入网络
计算相应的实际输出 O_p

$$O_p = F_n(\cdots (F_2(F_1(X_p W^{(1)}) W^{(2)}) \cdots) W^{(n)})$$

- 向后传播

计算实际输出 O_p 与相应的理想输出 Y_p 的差
按极小化误差的方法反向传播调整权矩阵



CNN适用范围

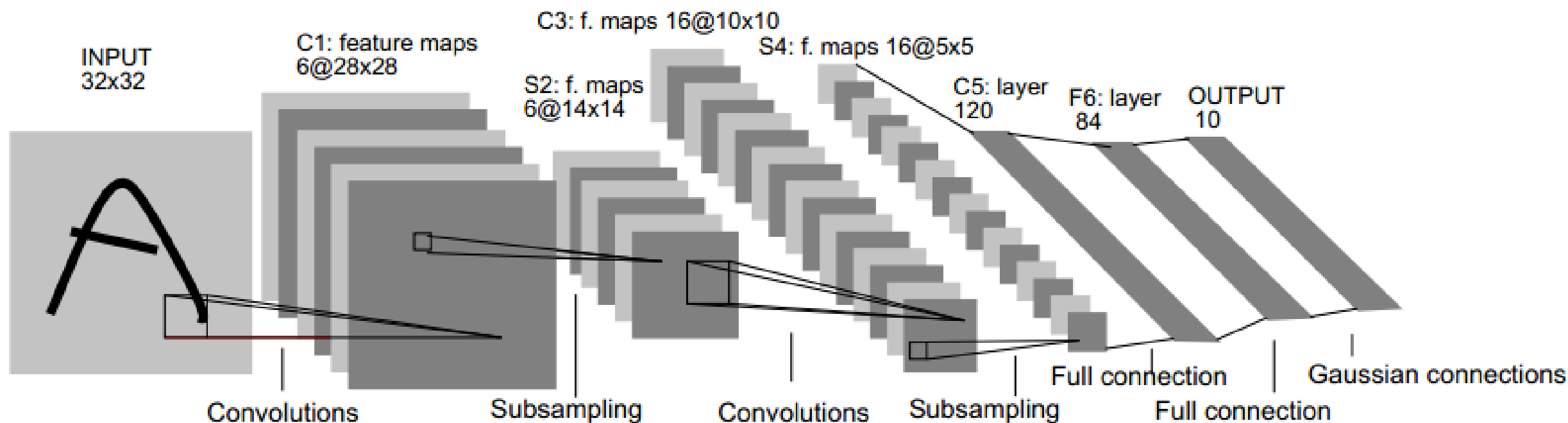
- 适用一维、二维及多维数据：距离较近数据的比距离较远的数据关联性更强(局部空间模式)
- 卷积神经网络只能捕捉数据中的局部“空间”模式。如果数据不能被转换成类似图像的数据，则不适用卷积神经网络。
 - 如果数据在交换任何列之后仍然同样有用，那么就不适合使用卷积神经网络
 - 例如如下客户数据

A	22	1A	a@a	1	aa	a1.a	123	aa1
B	33	2B	b@b	2	bb	b2.b	234	bb2
C	44	3C	c@c	3	cc	c3.c	345	cc3
D	55	4D	d@d	4	dd	d4.d	456	dd4
E	66	5E	e@e	5	ee	e5.e	567	ee5
F	77	6F	f@f	6	ff	f6.f	678	ff6
G	88	7G	g@g	7	gg	g7.g	789	gg7
H	99	8H	h@h	8	hh	h8.h	890	hh8
I	111	9I	i@i	9	ii	i9.i	901	ii9

Name, age,
address, email,
purchases,
browsing activity,...

CNN应用实例：文字识别系统LeNet-5

- 当年美国大多数银行就是用它来识别支票上面的手写数字的。

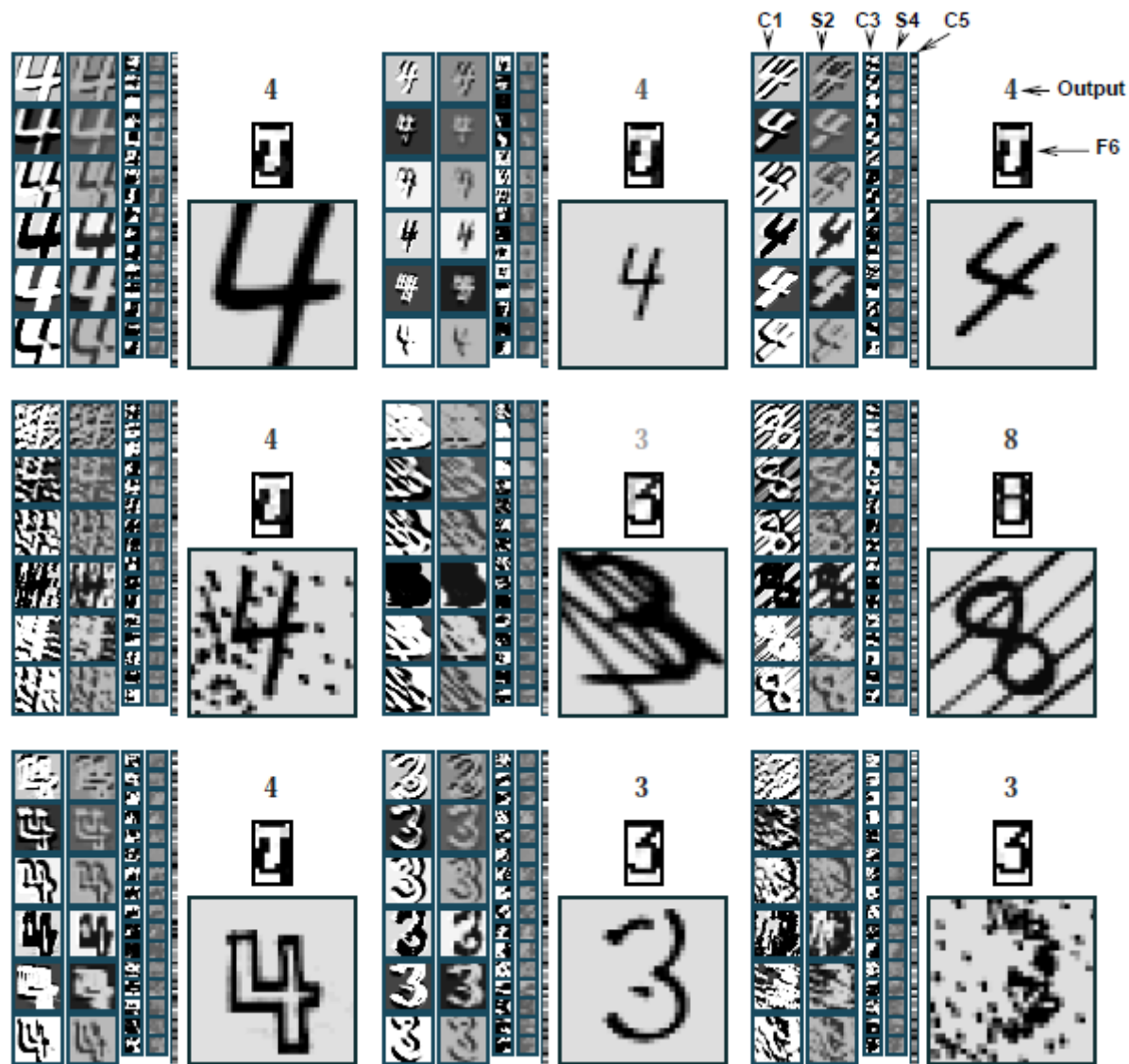


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X		X	X	X	X	X	X	X	X		X
5				X	X	X			X	X	X	X		X	X	X

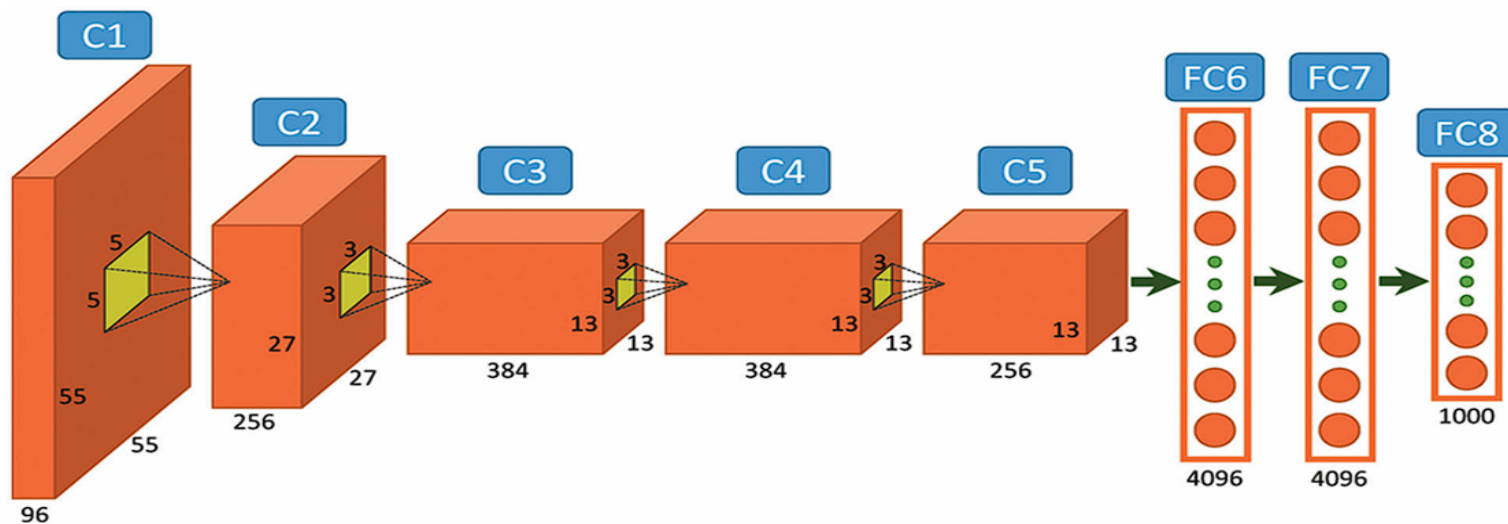
TABLE I

EACH COLUMN INDICATES WHICH FEATURE MAP IN S2 ARE COMBINED
BY THE UNITS IN A PARTICULAR FEATURE MAP OF C3.

文字识别系统LeNet-5



AlexNet

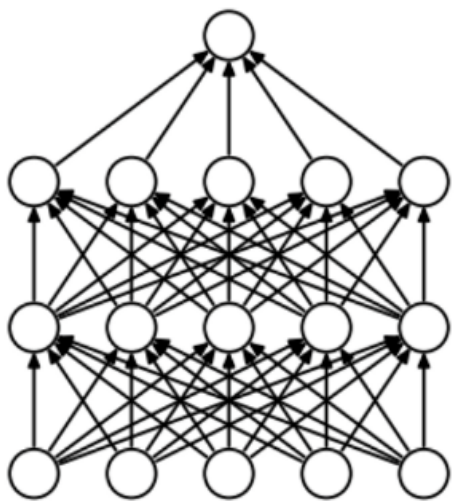


该架构(2012年ImageNet冠军)是首次将 ImageNet 分类准确率进行大幅提升的深度网络之一。它由 5 个卷积层和 3 个全连接层组成。它使用 ReLu（整流线性单元）来处理非线性部分，而不是 Tanh 或 Sigmoid 函数。该架构解决的另一个问题是通过在每个 FC 层后使用 Dropout 层来减少过拟合。使用两块 3GB 内存的 NVIDIA GTX 580 显卡训练。

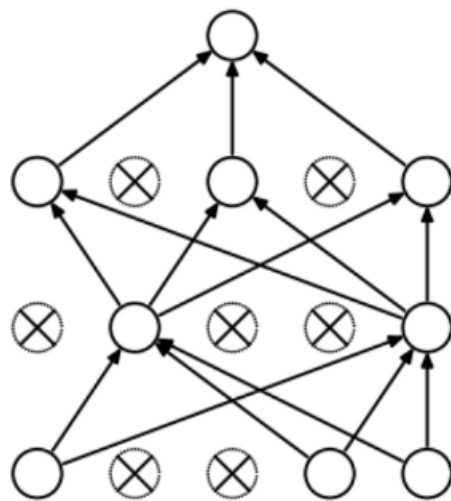
ImageNet：2009 年，ImageNet 数据集发布，要求研究人员从 100 万个示例中学习模型，每个示例来自 1000 个不同的对象类别。

AlexNet

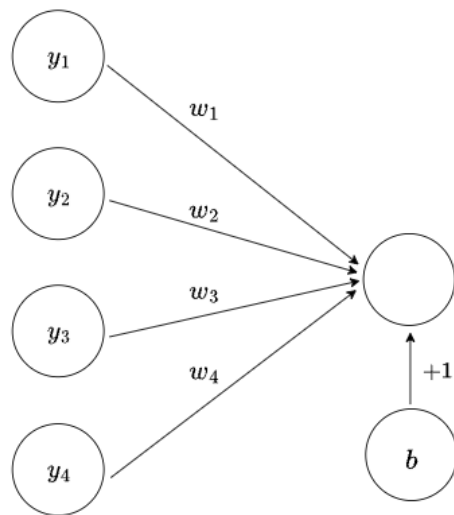
■ Dropout:



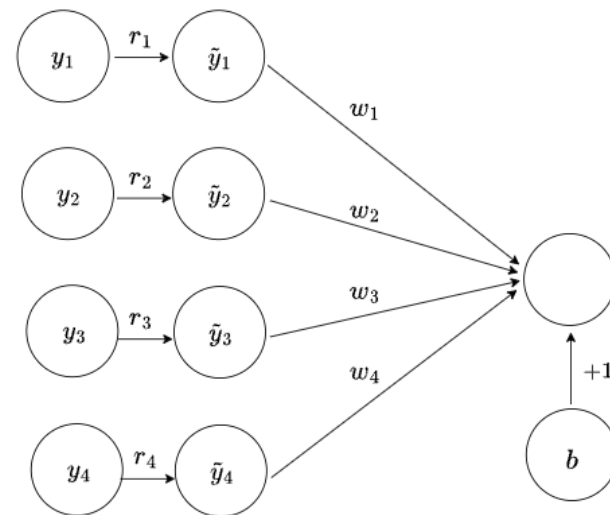
(a) Standard Neural Net



(b) After applying dropout.



(a) Standard Network



(b) Dropout Network

$r^l = \text{Bernoulli}(p)$ 伯努利分布

$$\tilde{y}^l = r^l y^l$$

$$z^{(l+1)} = w^{(l+1)} y^l + b^{(l+1)}$$

$$y^{(l+1)} = f(z^{(l+1)})$$

(a) Standard Network

$$z^{(l+1)} = w^{(l+1)} \tilde{y}^l + b^{(l+1)}$$

$$y^{(l+1)} = f(z^{(l+1)})$$

(b) Dropout Network

AlexNet

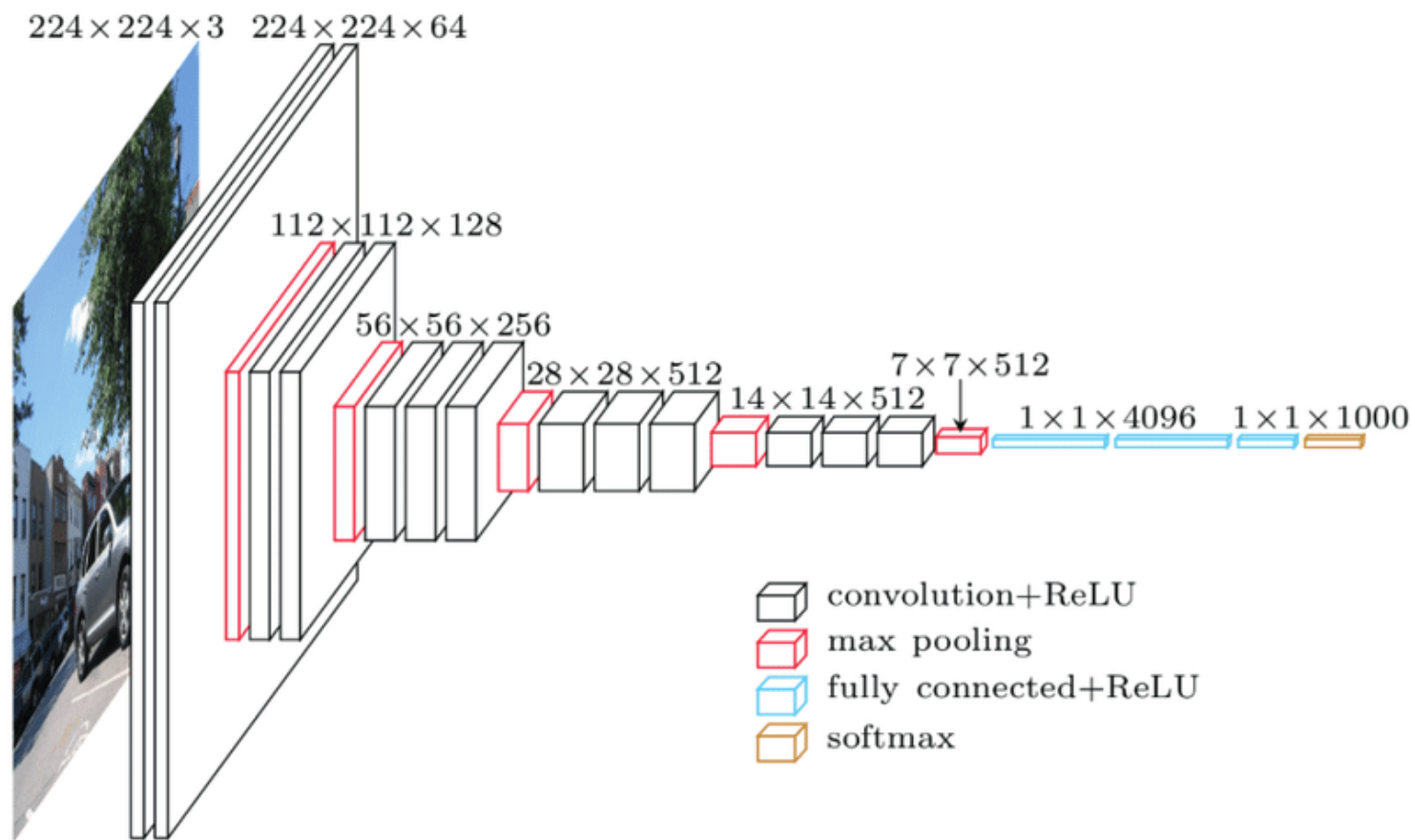
Dropout 如何工作？

训练期间：在每次训练迭代中，**dropout** 会随机禁用网络中的部分神经元（例如 **50%**）。这实际上会创建一个新的、更小的神经网络，其中包含更少的神经元。因此，模型可以学习在不依赖任何特定神经元的情况下工作，从而减少过拟合。

测试期间：在测试或推理期间，所有神经元都处于活动状态。神经元的权重会根据 **dropout** 率进行缩减。这种缩减通常以 **(1 - dropout 率)** 为单位，以确保输出范围与训练期间相同。

- 例如，如果将 **dropout** 率设置为 **0.5**，则在训练期间，每次迭代中一半的神经元会被禁用。测试时，所有神经元都处于活动状态，但每个神经元的输出都会乘以 **0.5**，以平衡网络的响应。

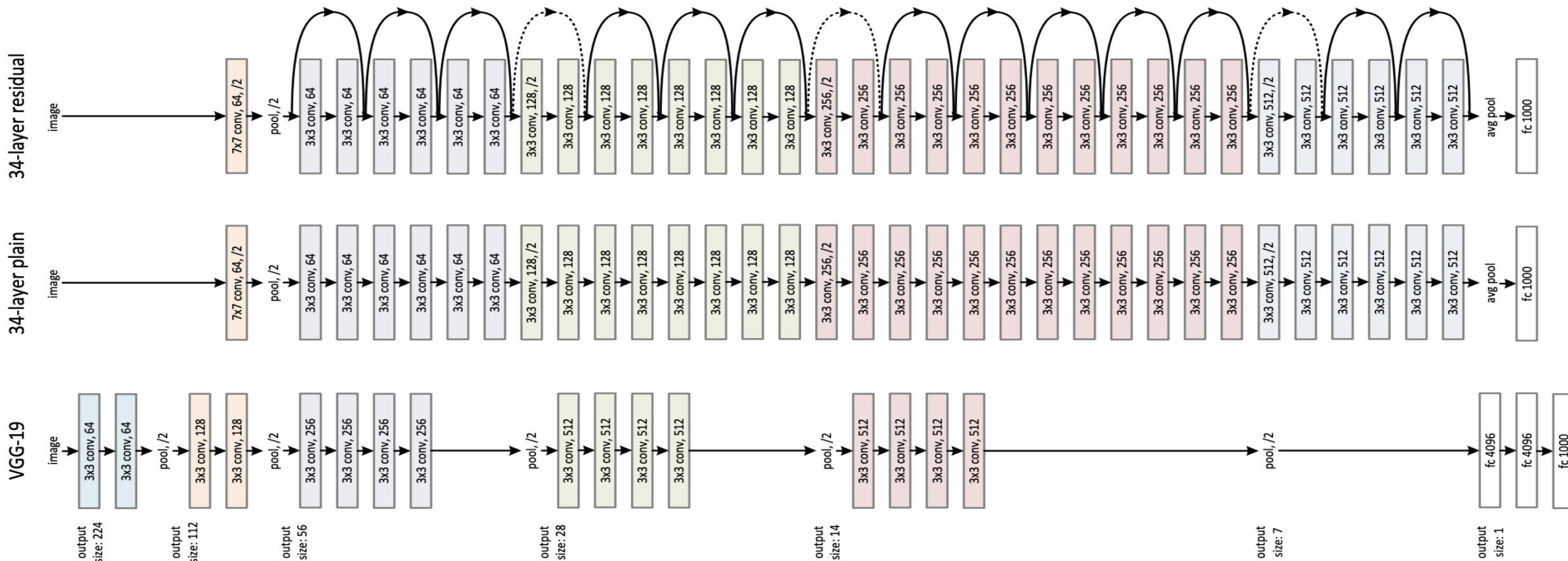
VGG16



在VGGNet中，卷积核大小为3*3，卷积步长为1，池化核为2*2，步长为2。VGGNet最明显的改进是减小卷积核大小，增加卷积层数。

2014年ImageNet冠军，准确率92.7%。它将AlexNet较大的卷积核（第一和第二卷积层中卷积核大小分别为 11x11 和 5x5 ）替换为多个 3X3 卷积核。在给定感受野的情况下，多个堆叠的卷积核比大卷积核效果更好，使其能够以更低的成本学习更复杂的特征

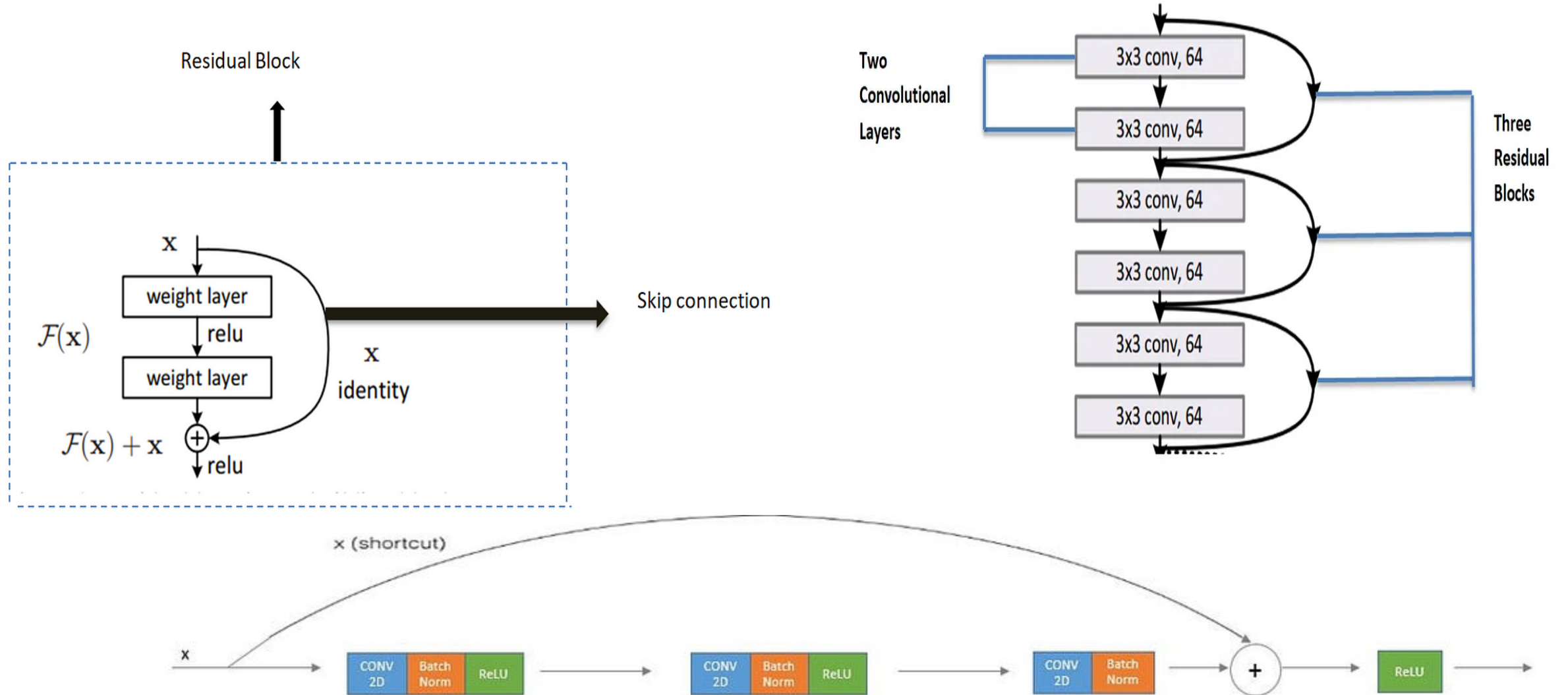
Residual Networks



ResNet（残差神经网络）架构由何凯明等人2015年提出。Resnet在2015年ImageNet图像分类任务中以**3.57%**的**Top 5**错误率遥遥领先于其他架构。网络架构基于普通的**34**层卷积网络，并借鉴了**VGG19**网络，在此之上增加了跳跃链接

Residual Networks

■ One residual block



Residual Networks

- 深度神经网络面临的挑战

- 神经网络中的退化问题:

- 随着神经网络深度的增加，网络在训练数据上的性能达到饱和，然后开始下降的现象。退化问题是深度网络的核心问题，因为它限制了更深层次的网络更能够提取复杂和抽象特征。我们可以将其视为双重问题
 - **性能平台期**: 随着神经网络层数的增加，训练误差趋于饱和并停止改善。这意味着额外的层对减少训练误差没有任何显著的好处。
 - **准确度下降**: 添加更多层后，验证集上的错误开始增加，测试数据集的性能变差。
 - 本质仍为**梯度消失/爆炸**

Residual Networks

- 为什么能解决梯度退化问题：

- 如果深层模型在浅层模型的基础上加入的层都是恒等映射（identity mapping）的话，则至少可以保证深层模型的误差不会比浅层模型的误差大。

$$\text{即 } x = F(x) + x$$

- 残差结构解决了传统网络中恒等映射难以拟合的问题：使用残差结构的多个非线性层如果要拟合恒等映射的话，只需要使 $F(x)=0$ 即可，拟合难度大大下降。
- 通过引入“跳跃连接”，使得信息可以直接从一个层跳到另一个层，而不需要通过所有的非线性层。这种设计允许梯度在网络中更容易地流动，从而解决了梯度消失和爆炸的问题。
- 在训练初期，由于层数增加，导致输入数据在网络中流动时，数据特征损失严重，网络无法有效学习数据特征，通过引入“跳跃连接”，使得信息可以直接从一个层跳到另一个层，而不需要通过所有的非线性层，有助于后续层学习到有效的数据特征。

CNN总结

- 卷积网络的核心思想：
 - 将局部感受野、权值共享以及时间或空间采样这三种结构思想结合起来获得了某种程度的位移、尺度、形变不变性。
- 层间联系和空间信息的紧密关系，使其适于图像处理和理解。
 - 图像和网络的拓扑结构能很好的吻合
- 避免了显式的特征抽取，而隐式地从训练数据中进行学习
 - 特征提取和模式分类同时进行，并同时在训练中产生；
 - 权重共享可以减少网络的训练参数，使神经网络结构变得更简单，适应性更强。
- CNN的超参数：
 - 卷积层数、池化层数、全链接层数、卷积核数量大小、池化核大小等需视具体

Deep Learning的常用模型

- Recurrent Neural Networks, RNN
- Long Short Term Memory, LSTM

序列数据建模(modeling sequences)

■ 序列数据的特点

- 改变序列的顺序会改变其含义或其所代表的模式。
 - 如自然语言句子：猫捉老鼠，老鼠捉猫
- 序列中的元素通常与之前（有时是之后）出现的元素相关或依赖。依赖的强度和范围差异较大。
 - 短程依赖：当前单词强烈依赖于紧邻的单词。
 - 长程依赖关系：预测或理解某个时间步元素所需的信息可能早在很多步骤之前就出现了。
- 与结构化数据集不同，序列通常具有可变的长度。

序列数据建模(modeling sequences)

- 传统全链接神经网络不适合处理序列数据
 - 忽略序列信息
 - 序列信息丢失，对数据元素的顺序不敏感
 - 如句子极性分类：
 - 他做事，事倍功半。 他做事，事半功倍。
 - 会输出同样结果
 - 依赖关系混乱，如当前信息只依赖于历史信息
 - 输入长度固定，而序列数据的长度是可变的
 - 如设计网络输入长度为最长序列长度，输入维度会非常高，学习参数增多

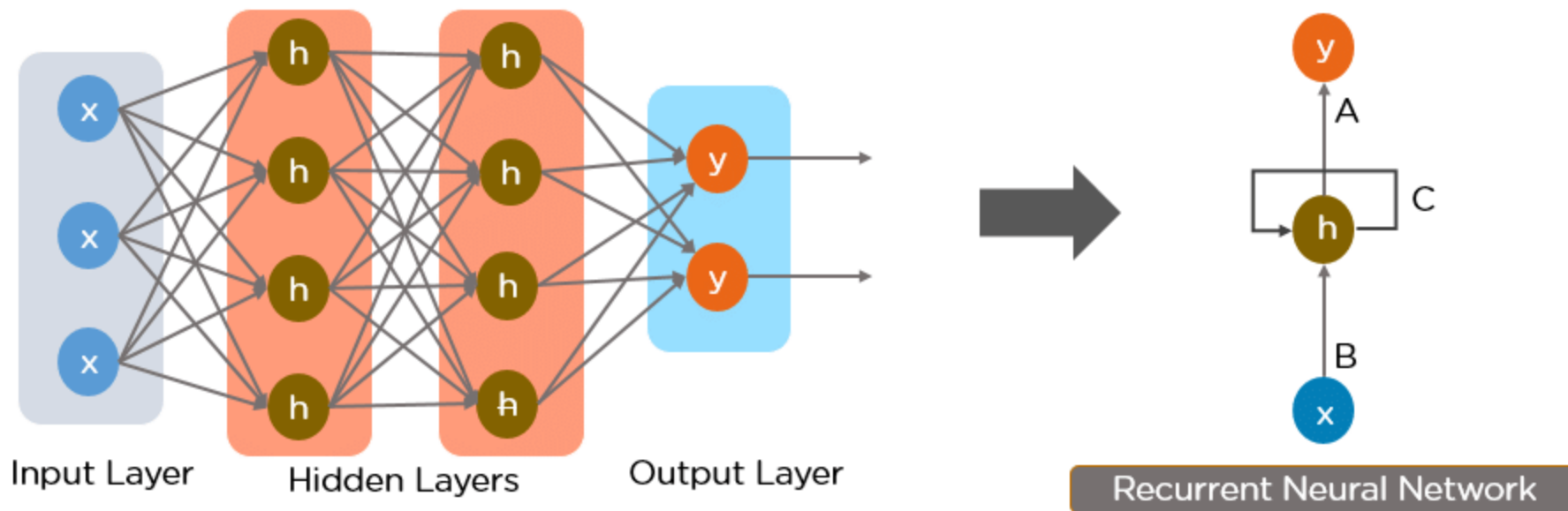
序列数据建模(modeling sequences)

- **循环神经网络 (RNN)** 是专门设计用于处理序列数据的神经网络，它通过内部**隐状态**保存先前输入的记忆。该记忆保留了历史每一步的信息，并用它来影响未来的预测结果。
 - RNN在不同的时间步骤之间共享参数，比传统的前馈网络（每一层都有自己的参数）更有效率。
 - 利用**时间反向传播 (BP through time)**进行训练，考虑每个时间步的误差进行权重学习，确保过去和当前状态都有助于输出正确结果。
- RNN是在自然语言处理领域中最先被用起来的，用来为语言模型建模 (Jordan, 80年代末)

Recurrent Neural Networks, RNN

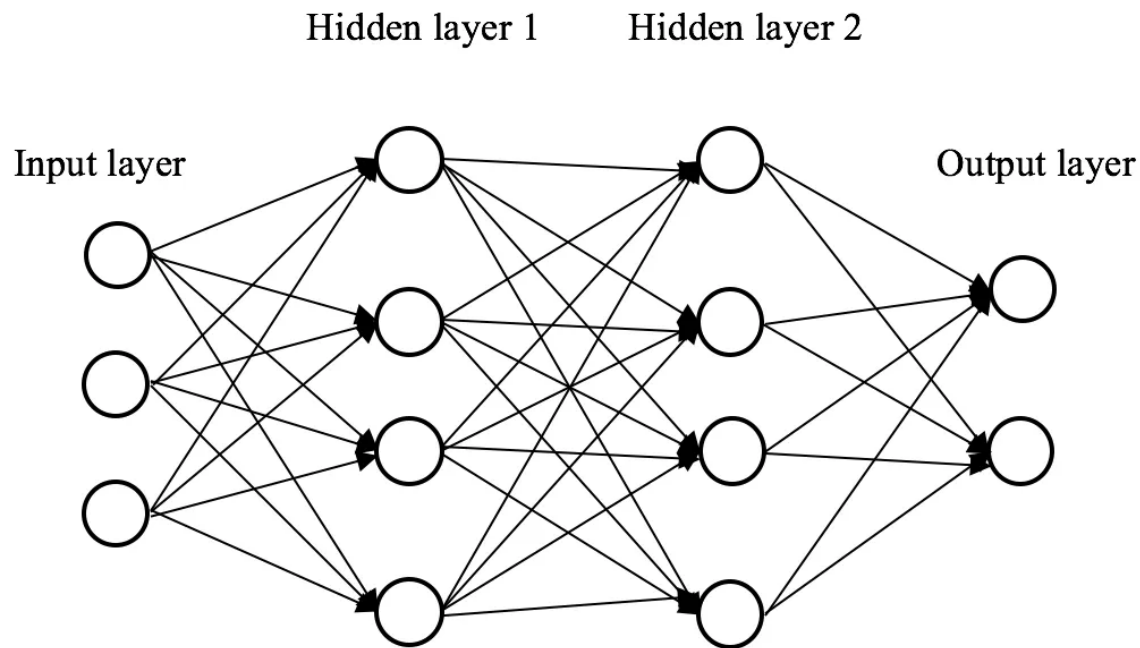
■ 如何记住历史信息

- 标准神经网络中的所有输入和输出都是相互独立的。然而，在某些情况下，例如预测短语的下一个单词时，记住前面的单词是必需的，**RNN** 使用隐藏层反馈实现历史记忆。
 - 如：他上学迟到了，老师批评了？
- 该反馈将隐藏层上一步的输出用作下一步的输入，从而与前一步的数据建立连接和记忆。这样，**RNN** 能够生成序列数据间的依赖关系或连续的上下文依赖关系。

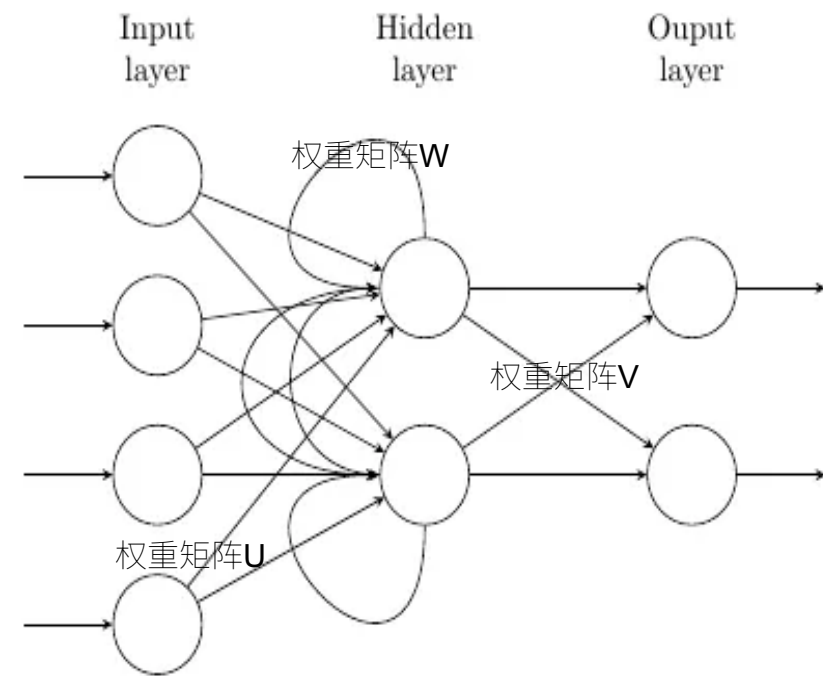


Recurrent Neural Networks, RNN

■ RNN的反馈结构：未按时序展开



前馈神经网络

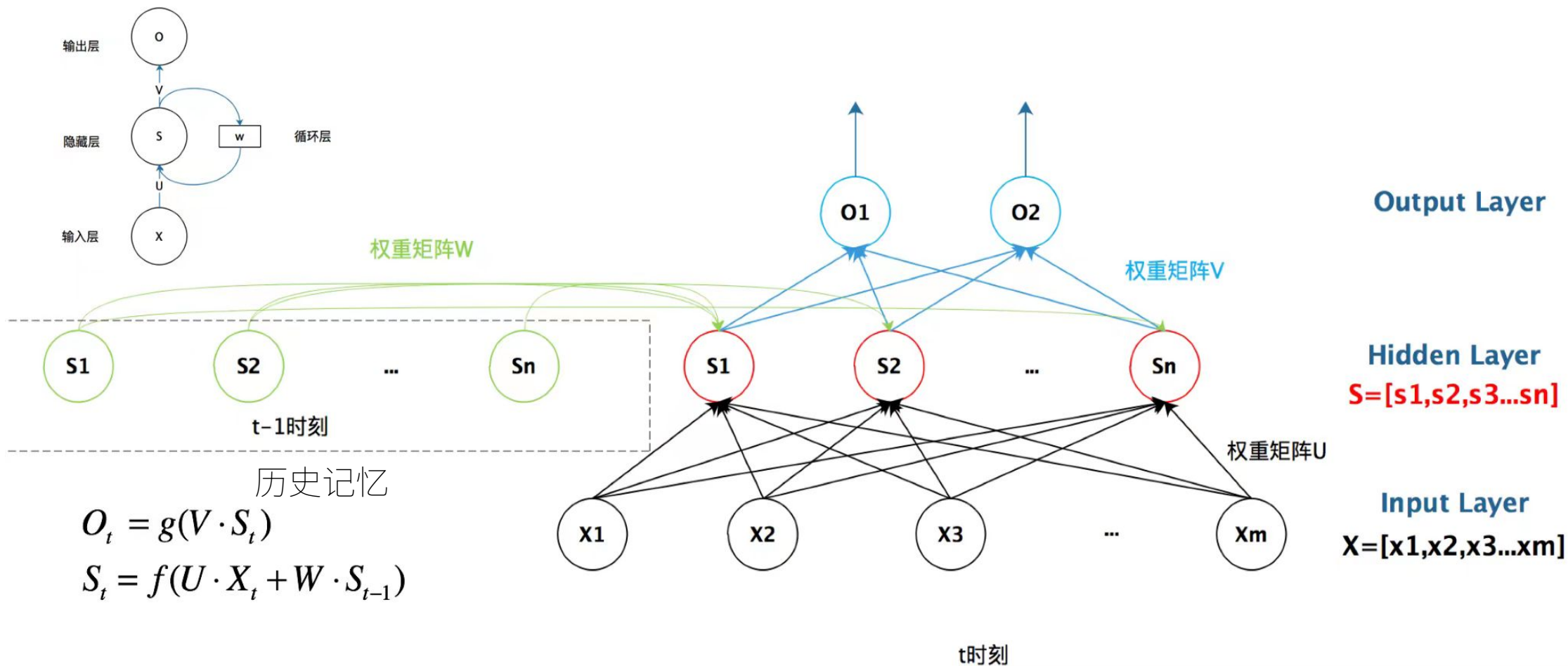


RNN

注意：隐藏层反馈的时序

Recurrent Neural Networks, RNN

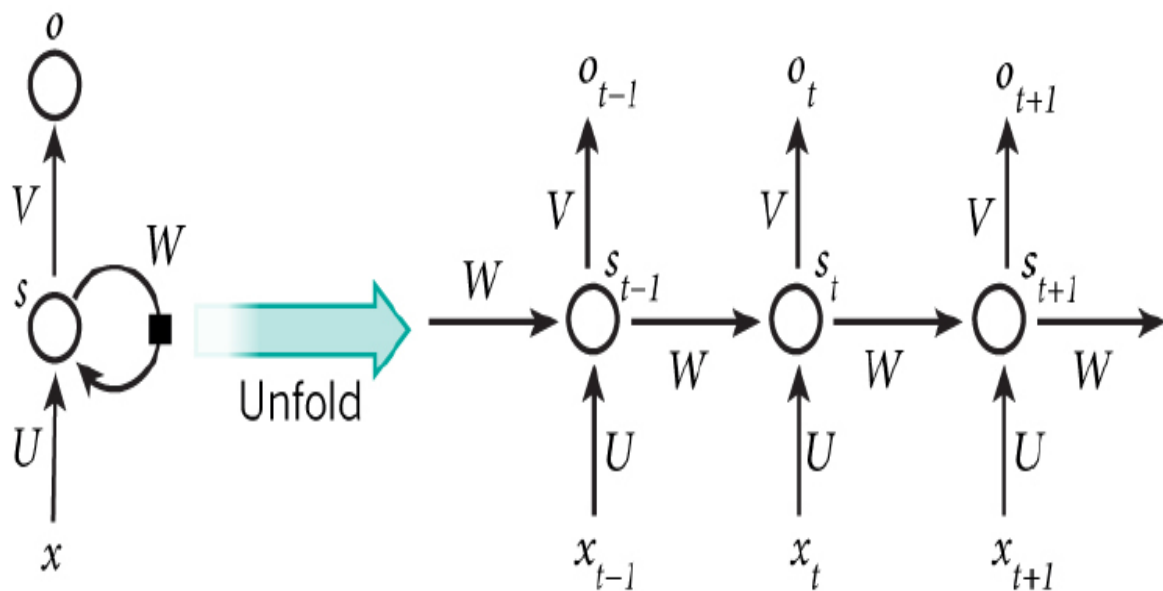
- Hidden Layer会有连向下一时间Hidden Layer的边， 权为W



Recurrent Neural Networks, RNN

■ RNN按时序展开

- s 隐藏层的状态向量
 - 包含全部隐藏层节点的状态，节点数与向量 s 的维度相同
- U 是输入层到隐藏层的权重矩阵
- O 输出层向量
- V 是隐藏层到输出层的权重矩阵
- W 隐藏层反馈权重矩阵，各时间步共享
- s_t 取决于 x_t , s_{t-1}



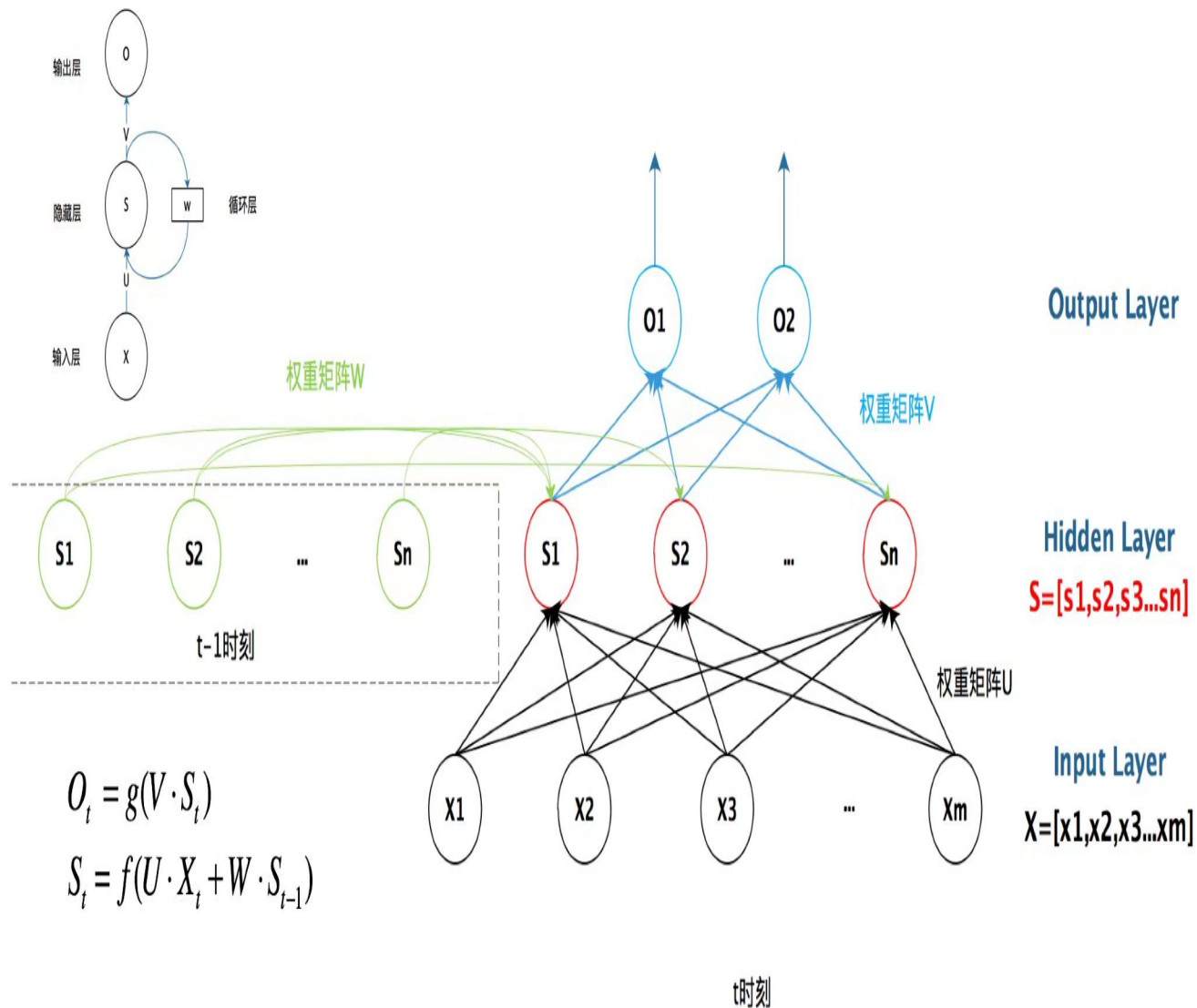
Recurrent Neural Networks, RNN

■ RNN的计算过程

$$\begin{aligned}o_t &= g(Vs_t) \\s_t &= f(Ux_t + Ws_{t-1})\end{aligned}$$

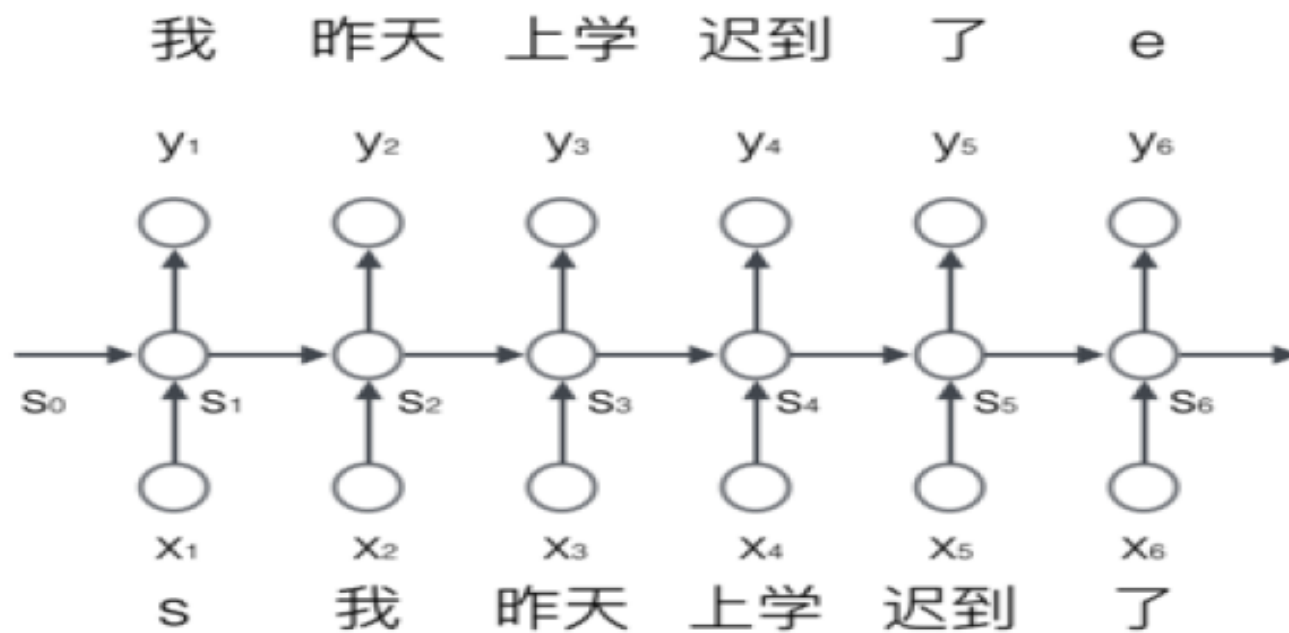
$$\begin{bmatrix} s_1^t \\ s_2^t \\ \vdots \\ s_n^t \end{bmatrix} = f\left(\begin{bmatrix} u_{11} & u_{12} & \dots & u_{1m} \\ u_{21} & u_{22} & \dots & u_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ u_{n1} & u_{n2} & \dots & u_{nm} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n1} & w_{n2} & \dots & w_{nn} \end{bmatrix} \begin{bmatrix} s_1^{t-1} \\ s_2^{t-1} \\ \vdots \\ s_n^{t-1} \end{bmatrix} \right)$$

$$\begin{aligned}o_t &= g(Vs_t) \\&= Vf(Ux_t + Ws_{t-1}) \\&= Vf(Ux_t + Wf(Ux_{t-1} + Ws_{t-2})) \\&= Vf(Ux_t + Wf(Ux_{t-1} + Wf(Ux_{t-2} + Ws_{t-3}))) \\&= Vf(Ux_t + Wf(Ux_{t-1} + Wf(Ux_{t-2} + Wf(Ux_{t-3} + \dots))))\end{aligned}$$



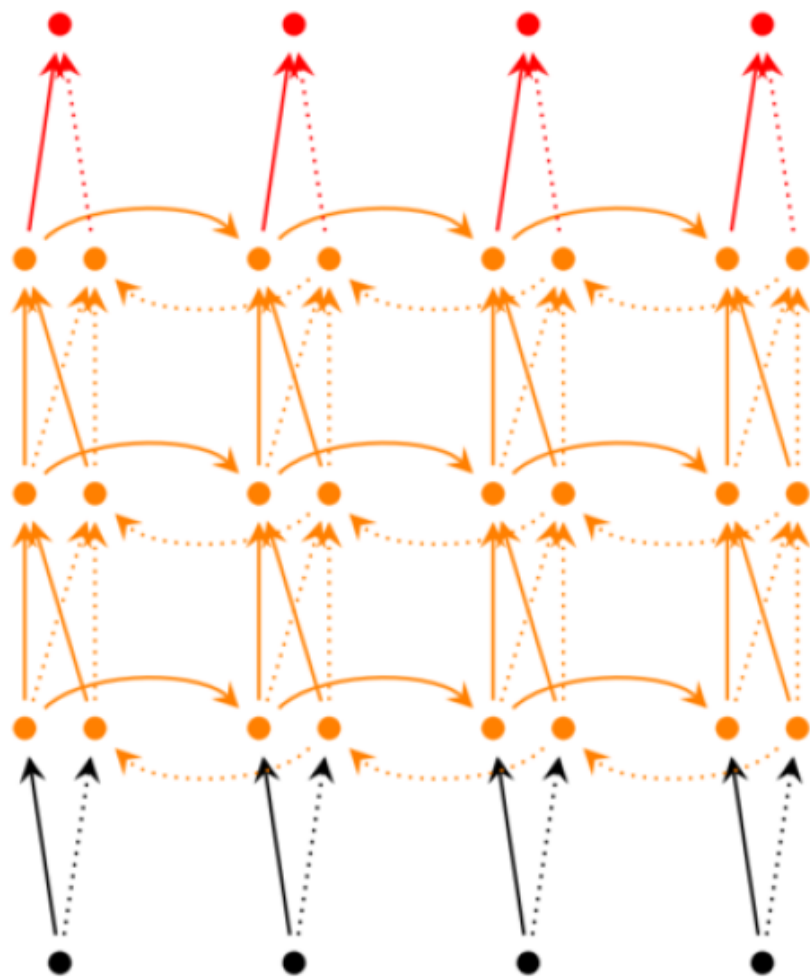
Recurrent Neural Networks, RNN

- 输入、输出：一般来说，RNN每一时刻的输入和输出是不一样的
 - 例如序列预测：将序列元素依次传入，每个序列元素再对应不同的输出



深度RNN

- 堆叠两个以上的隐藏层，这样就得到了深度循环神经网络。



RNN 训练: Back Propagation Through Time (BPTT)

■ RNN的权值学习

$$E = \sum_{t=0}^T E_t \quad \frac{\partial E}{\partial W} = \sum_{t=1}^T \frac{\partial E_t}{\partial W} W \leftarrow W - \eta \frac{\partial E}{\partial W}$$

$$\frac{\partial E}{\partial w_x} = \sum_{k=0}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial s_t} \left(\prod_{j=k+1}^t \frac{\partial s_j}{\partial s_{j-1}} \right) \frac{\partial s_k}{\partial w_x}$$

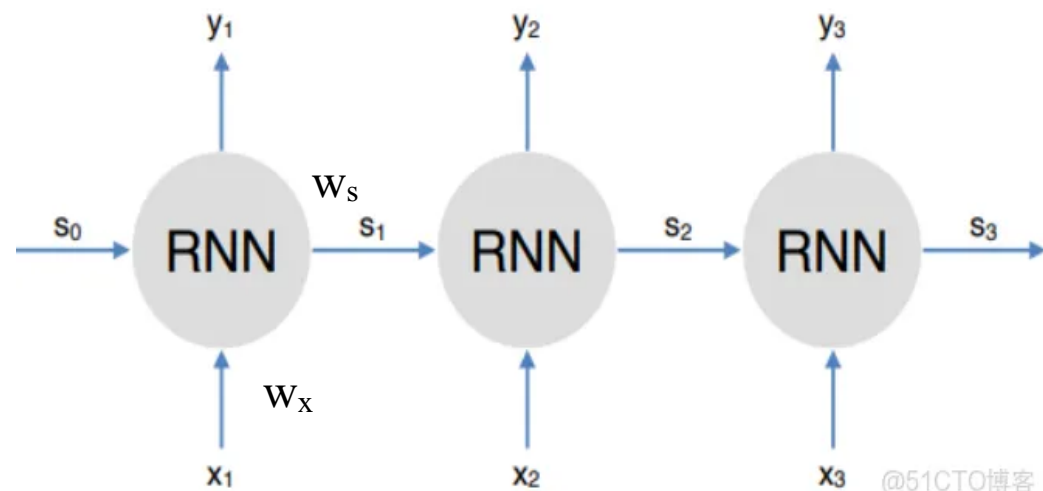
$$\begin{aligned} \frac{\partial s_j}{\partial s_{j-1}} &= \tanh'(w_s s_{j-1} + w_x x_j) \cdot \frac{\partial}{\partial s_{j-1}} [w_s s_{j-1} + w_x x_j] \\ &= \tanh'(w_s s_{j-1} + w_x x_j) \cdot w_s \end{aligned}$$

$\tanh'$ 在0,1之间, 如果 w_s 也在0,1之间则容易产生梯度消失

$$\begin{aligned} \frac{\partial E}{\partial w_x} &= \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial s_3} \frac{\partial s_3}{\partial w_x} + \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial w_x} + \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial w_x} \\ \frac{\partial E}{\partial w_s} &= \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial s_1} \frac{\partial s_3}{\partial w_s} + \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial w_s} + \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial w_s} \end{aligned}$$

(因为 s_3 是包含了 s_2 和 s_1 的)

例如: $t=3$, 序列长度为3, $E_3 = \frac{1}{2}(Y_3 - y_3)^2$



RNN梯度的膨胀或消散

- 训练长序列 (100 time steps) RNN，梯度很容易膨胀或消散

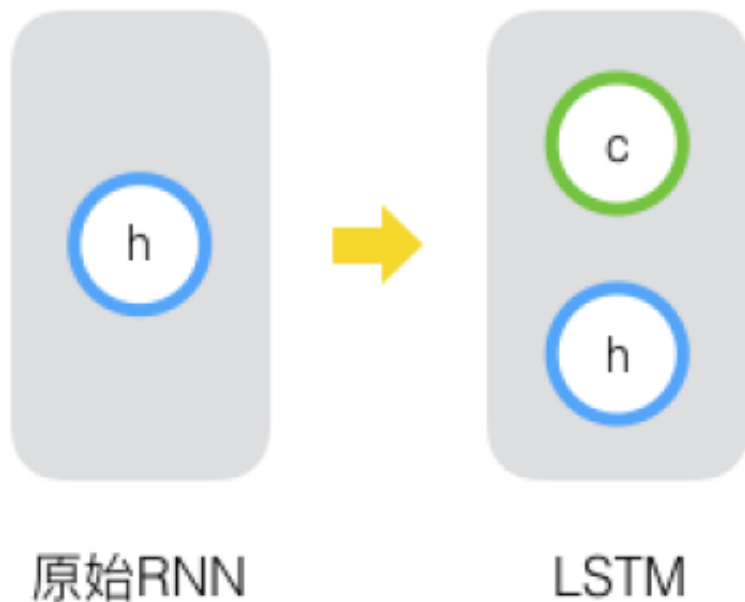
$$\begin{aligned}\frac{\partial s_j}{\partial s_{j-1}} &= \tanh'(w_s s_{j-1} + w_x x_j) \cdot \frac{\partial}{\partial s_{j-1}} [w_s s_{j-1} + w_x x_j] \\ &= \tanh'(w_s s_{j-1} + w_x x_j) \cdot w_s\end{aligned}$$

- $\tanh'$ 在0,1之间，如果 w_s 也在0,1之间则容易产生梯度消失
- RNN 很难记住长距离依赖关系 (长期记忆)
- 解决办法： Long Short Term Memory

Long Short Term Memory, LSTM

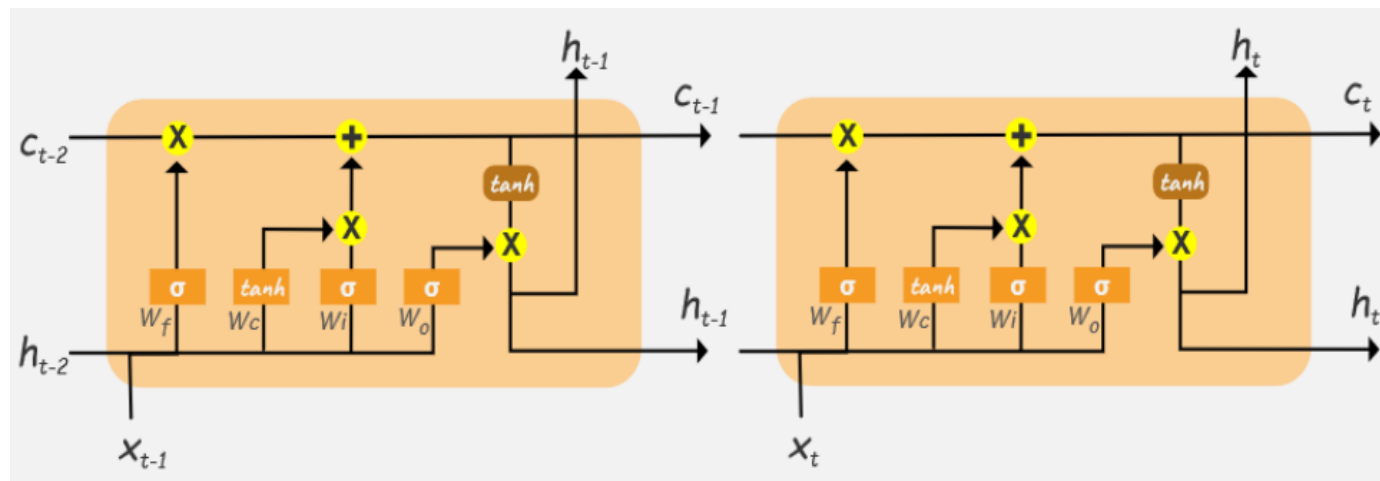
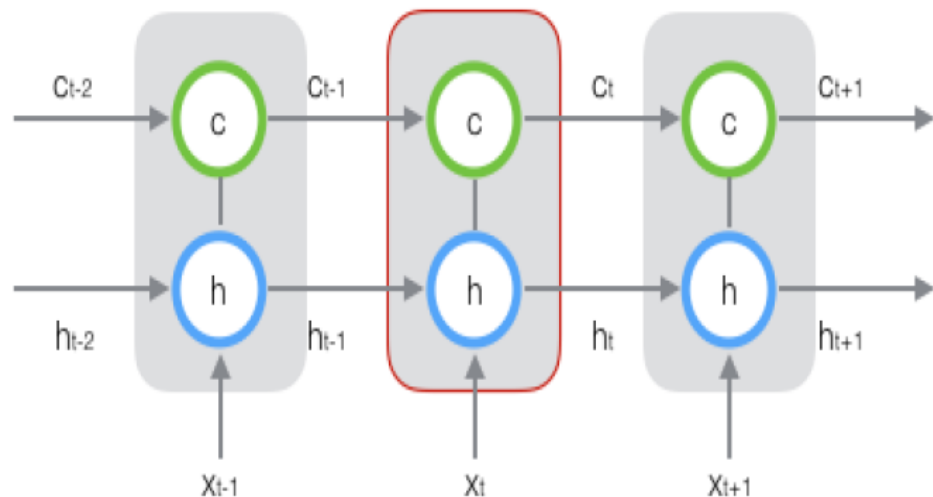
- 长短期记忆网络的解决思路：

- 原始RNN的隐藏层只有一个状态，即 h ，它对于短期的输入非常敏感，很难记忆长期依赖。那么，假如我们再增加一个状态，即 c ，让它来保存长期记忆， h 保存短期依赖，那么问题就解决了， c 也称作单元状态(cell state)
- Hochreiter & Schmidhuber (1997)



LSTM的时序展开

- LSTM按照时间维度展开：



- 在 t 时刻，LSTM的输入有三个：当前时刻网络的输入值 $\mathbf{x}_t$ 、上一时刻LSTM的输出值 $\mathbf{h}_{t-1}$ 、以及上一时刻的单元状态 $\mathbf{c}_{t-1}$ ；
- LSTM的输出有两个：当前时刻LSTM输出值 $\mathbf{h}_t$ 、和当前时刻的单元状态 $\mathbf{c}_t$

长短期记忆网络的门

- 门 (gate) 的概念

- 门实际上就是一层全连接层，它的输入是一个向量，输出是一个0到1之间的实数向量

$$g(\mathbf{x}) = \sigma(W\mathbf{x} + \mathbf{b})$$

- 门的使用

- 用门的输出向量按元素乘以需要控制的那个向量，实现门控制

- LSTM共有三个门：遗忘门、输入门、输出门

- LSTM用两个门来控制单元状态 $\mathbf{c}$ 的内容

- 遗忘门 (forget gate)，决定上一时刻的单元状态 $\mathbf{c}_{t-1}$ 有多少保留到当前时刻 $\mathbf{c}_t$
- 输入门 (input gate)，决定当前时刻网络的输入 $\mathbf{x}_t$ 有多少保存到单元状态 $\mathbf{c}_t$

- LSTM用输出门(output gate)控制当前时刻的短期记忆 $\mathbf{h}_t$

- 控制单元状态 $\mathbf{c}_t$ 有多少作为 $\mathbf{h}_t$

LSTM的长短期记忆形成

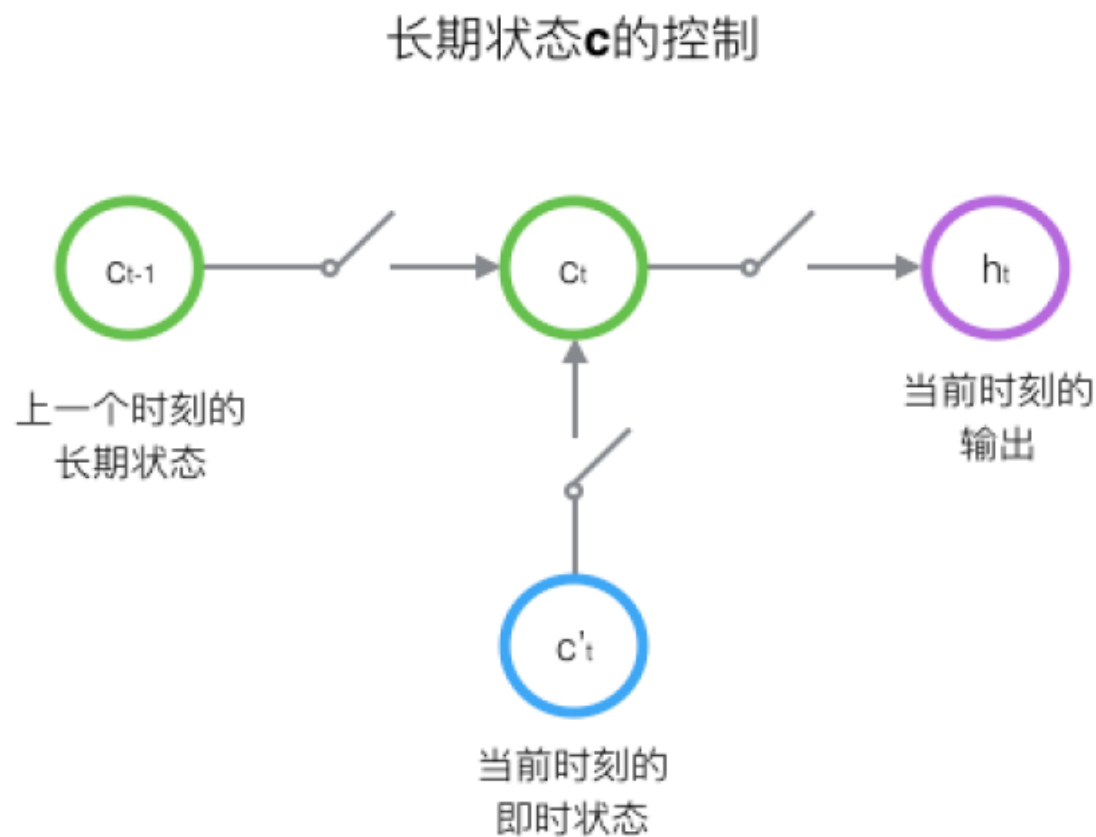
■ 控制长期状态 c

■ 使用三个控制开关

- 第一个开关，负责控制继续保存长期状态 c
- 第二个开关，负责控制把即时状态输入到长期状态 c

■ 控制短期状态 h_t

- 第三个开关，负责控制是否把长期状态 c_t 的一部分作为当前时刻的短期记忆 h_t 。

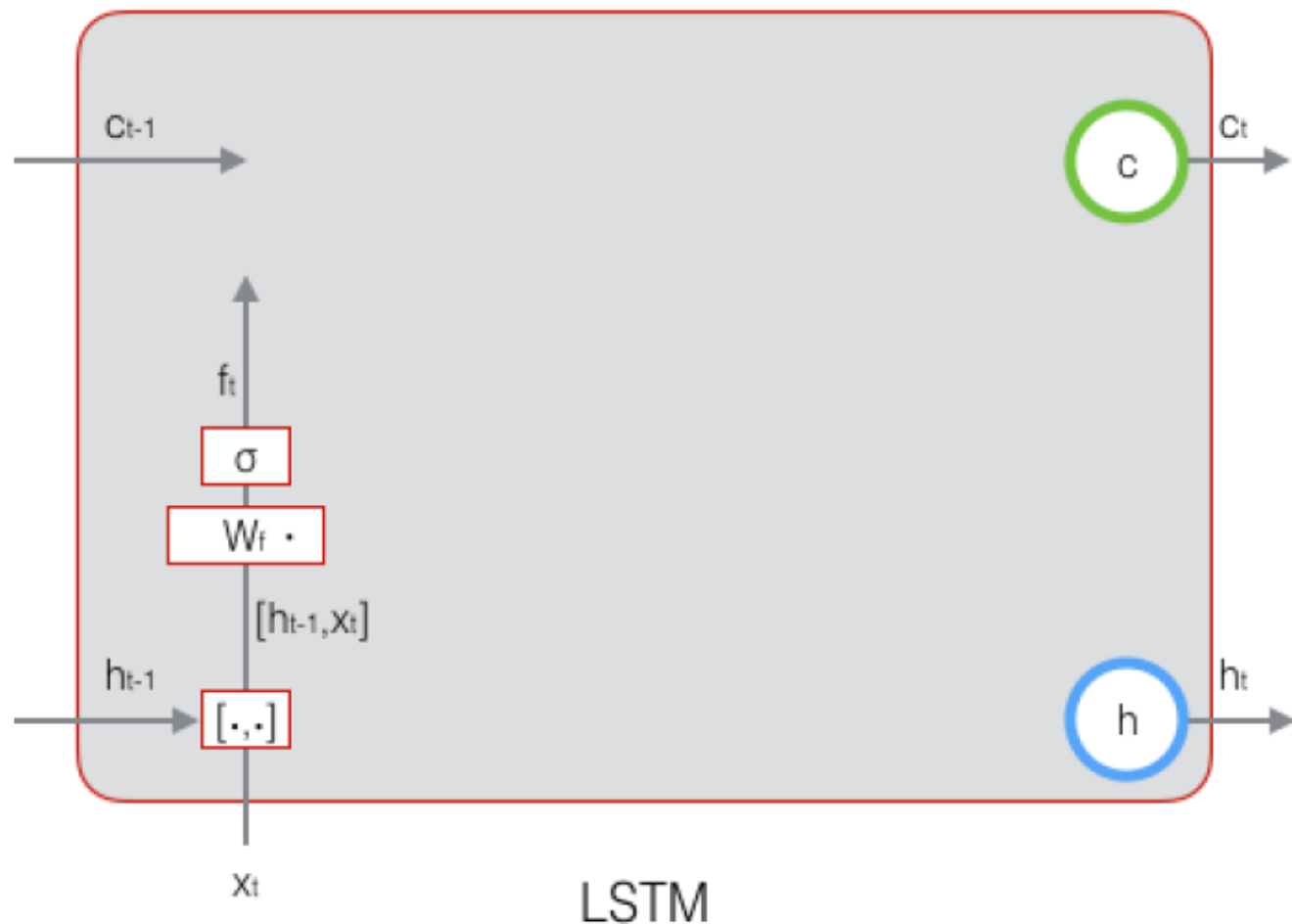


长短时记忆网络的计算过程

- 遗忘门: $\mathbf{f}_t = \sigma(W_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$

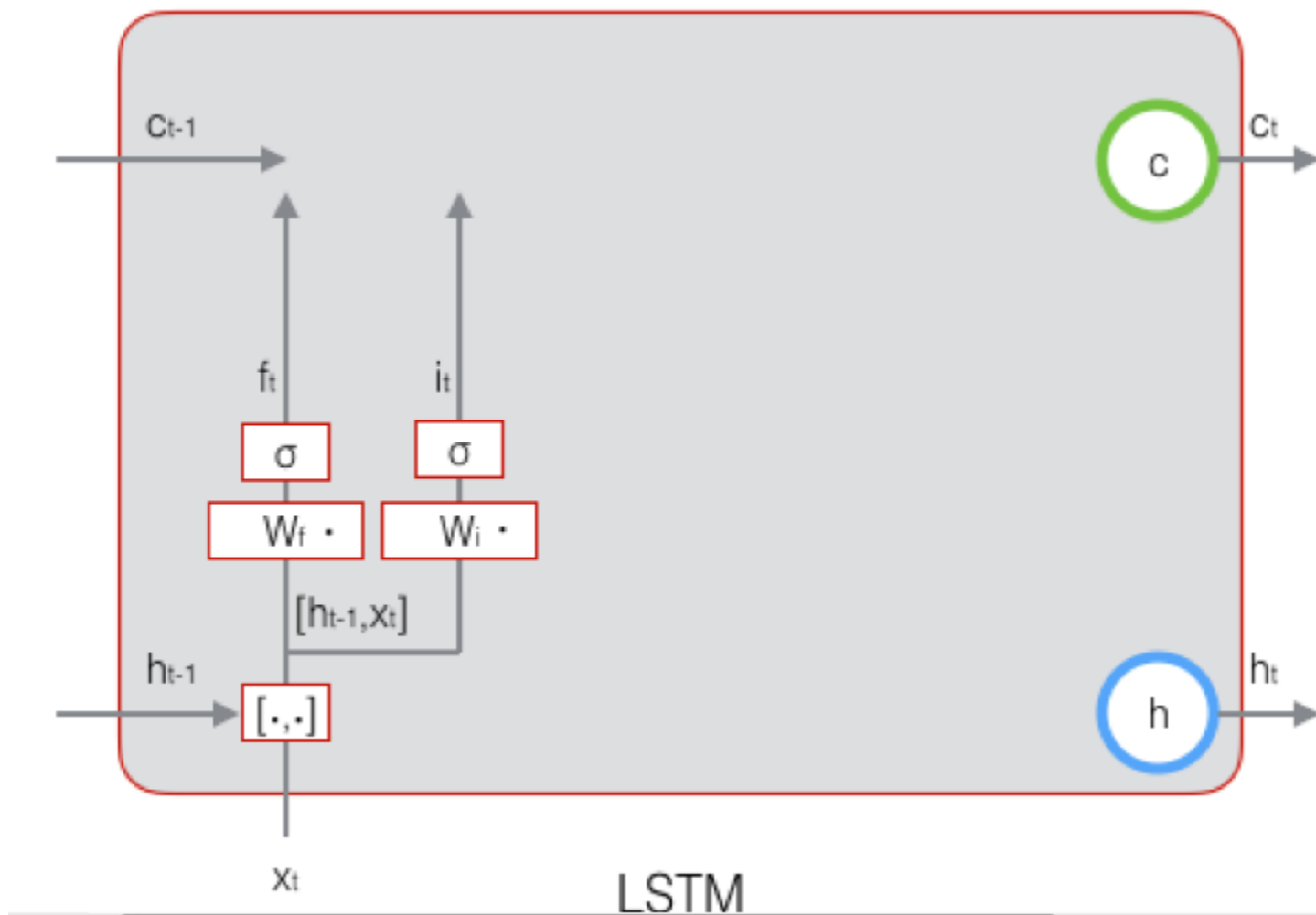
▪ σ 是sigmoid函数

$$\begin{aligned} [W_f] \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} &= [W_{fh} \quad W_{fx}] \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} \\ &= W_{fh} \mathbf{h}_{t-1} + W_{fx} \mathbf{x}_t \end{aligned}$$



长短时记忆网络的计算过程

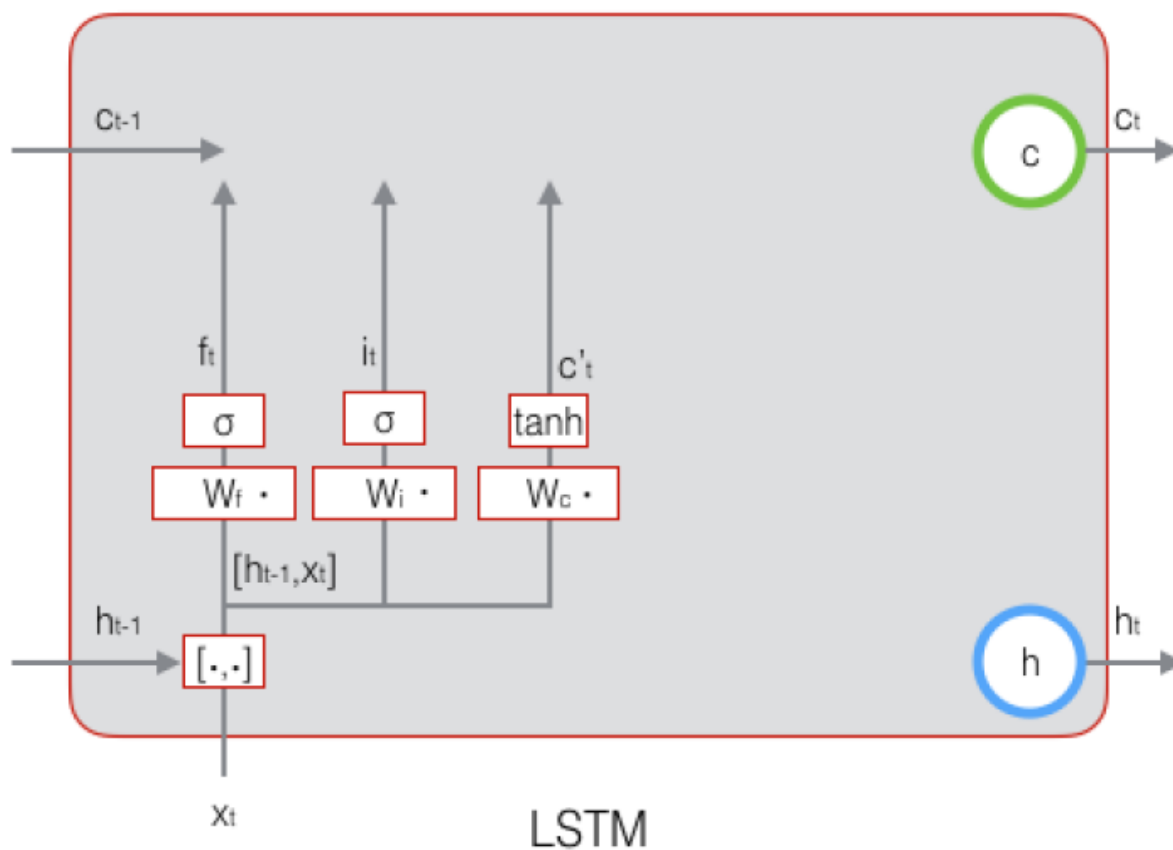
- 输入门: $\mathbf{i}_t = \sigma(W_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$



长短时记忆网络的计算过程

- 当前时刻的单元状态 $\tilde{\mathbf{c}}_t$ 由上一次的输出和本次输入决定：

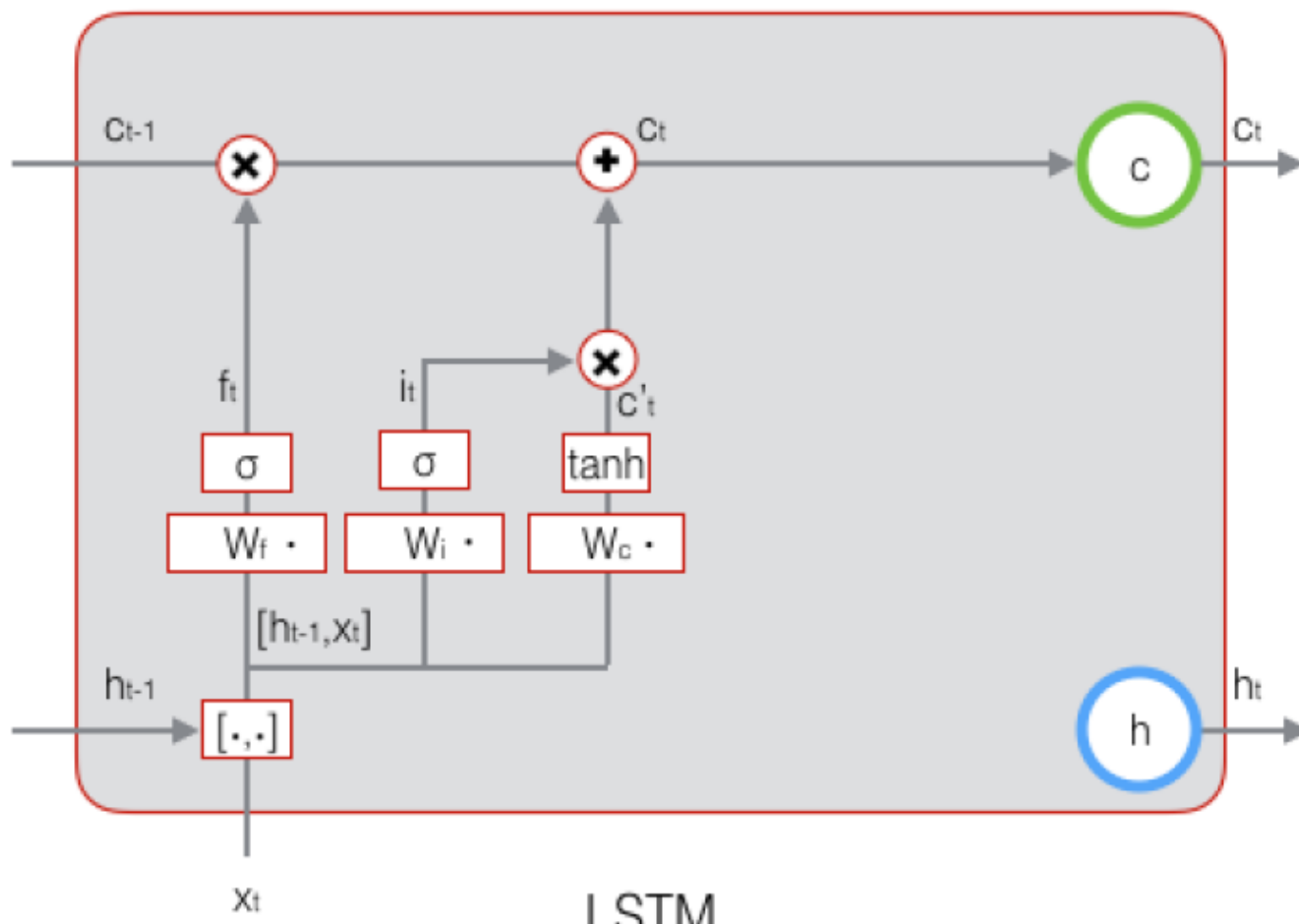
$$\tilde{\mathbf{c}}_t = \tanh(W_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$



长短时记忆网络的计算过程

- 当前时刻的单元状态
(长期记忆)
 - 由于遗忘门的控制，它可以保存很久很久之前的信息，由于输入门的控制，它又可以避免当前无关紧要的内容进入记忆。

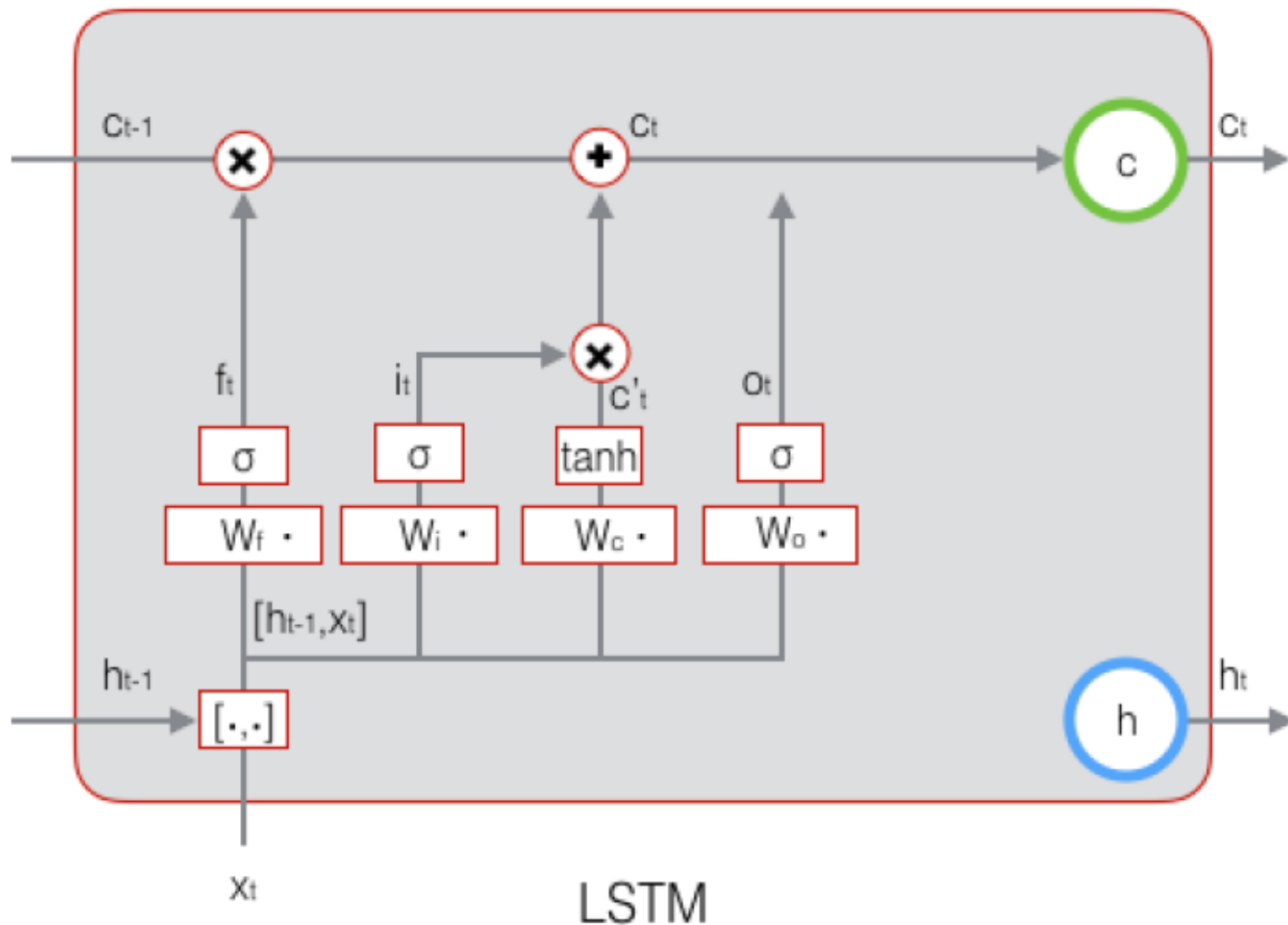
$$\mathbf{c}_t = \mathbf{f}_t \circ \mathbf{c}_{t-1} + \mathbf{i}_t \circ \tilde{\mathbf{c}}_t$$



长短时记忆网络的计算过程

- 输出门，它控制了长期记忆对当前输出(短期记忆)的影响

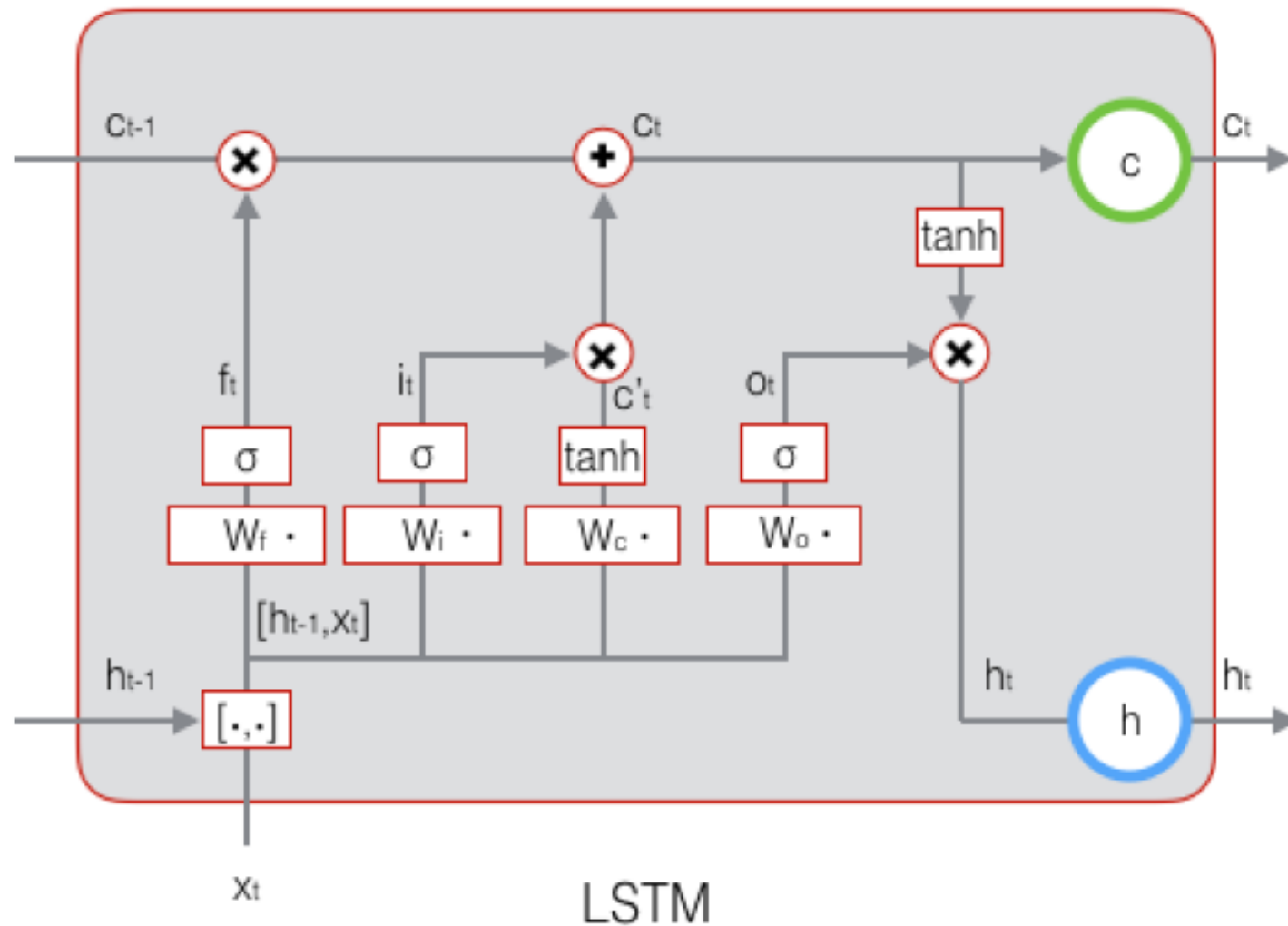
$$\mathbf{o}_t = \sigma(W_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$



长短时记忆网络的计算过程

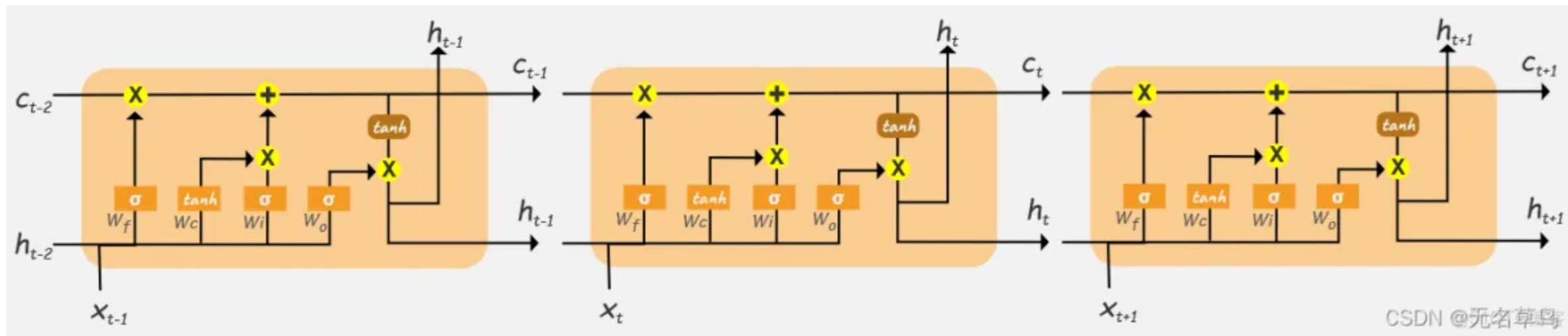
- 最终输出(短期记忆)，由输出门和单元状态共同确定的：

$$\mathbf{h}_t = \mathbf{o}_t \circ \tanh(\mathbf{c}_t)$$



长短期记忆网络的分析

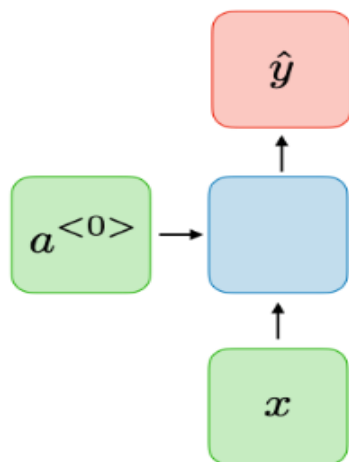
- 长期记忆C的迭代计算过程没有经过非线性神经网络层，能保存更长的依赖信息
- 长期记忆C与残差网中的跳跃链接有类似功能，使得信息可以直接从一个层跳到另一个层，而不需要通过所有的非线性层。这种设计允许梯度在网络中更容易地流动，从而避免了梯度消失和爆炸的问题。



Recurrent Neural Networks的应用类型

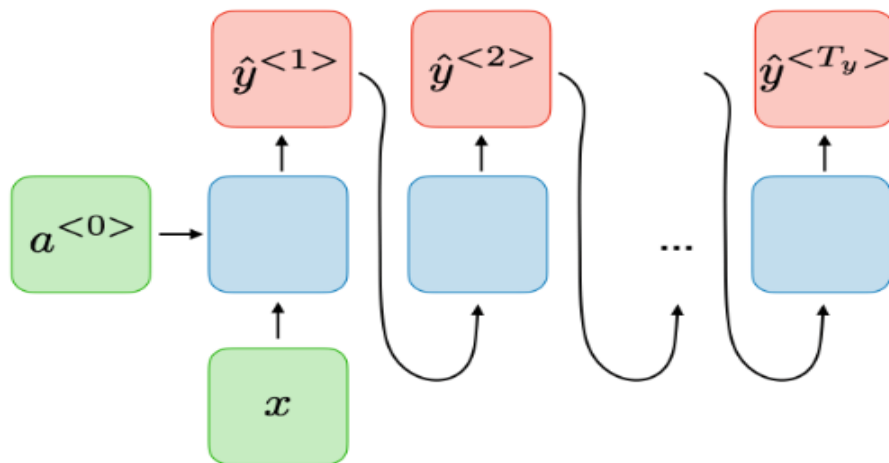
- 注： $a^{<0>}$ 初始时刻的隐状态变量

One-to-one
 $T_x = T_y = 1$



Traditional neural network

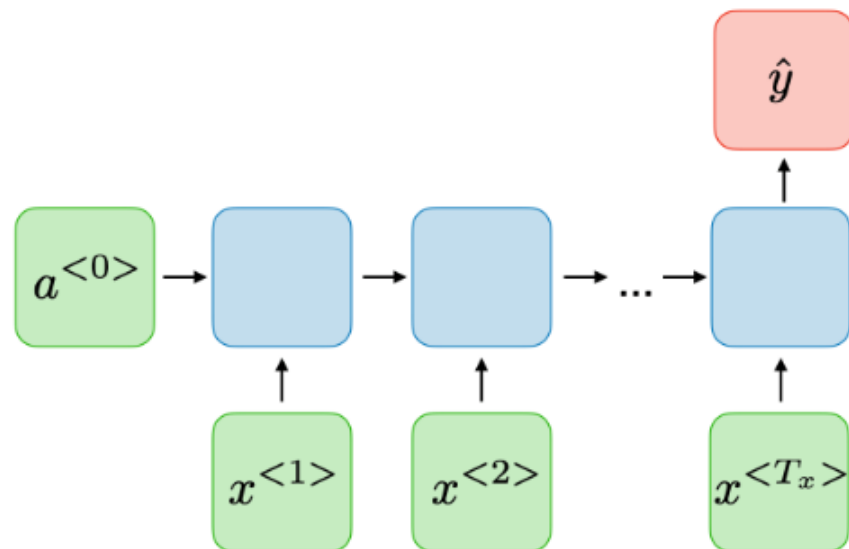
One-to-many
 $T_x = 1, T_y > 1$



Music generation

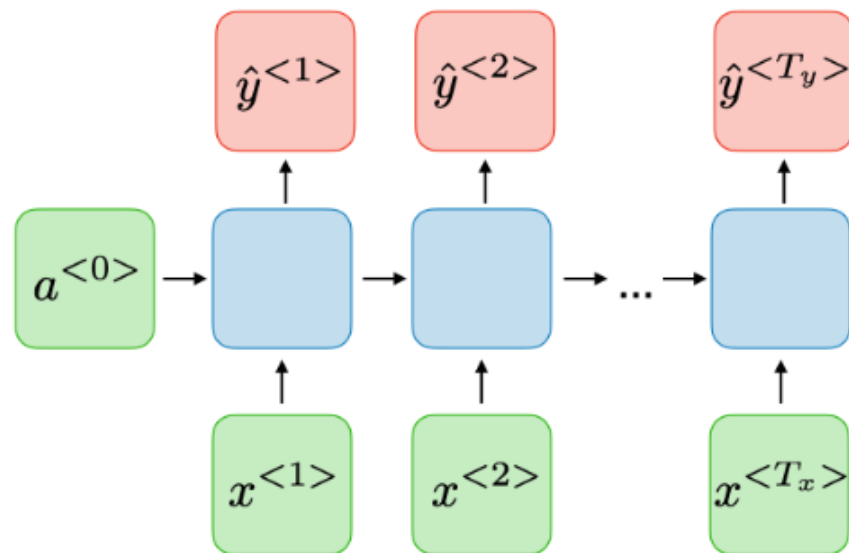
Recurrent Neural Networks的应用类型

Many-to-one
 $T_x > 1, T_y = 1$



Sentiment classification

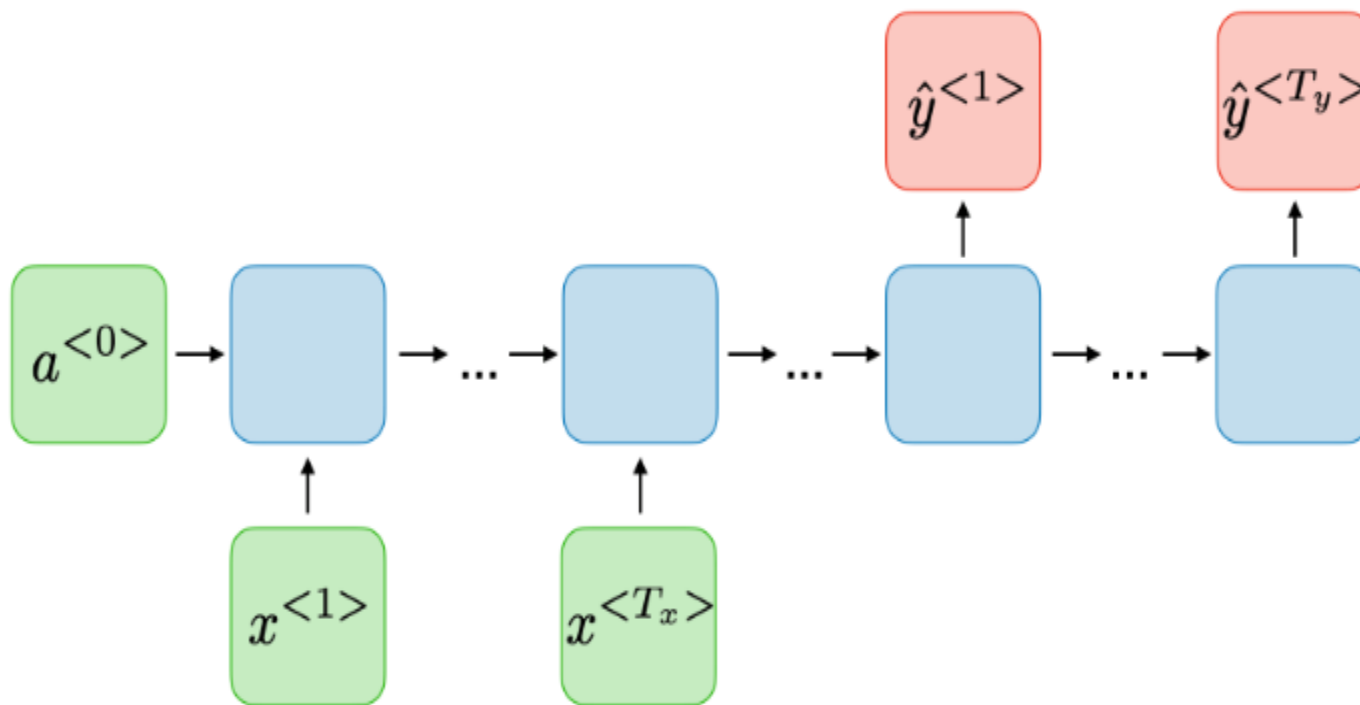
Many-to-many
 $T_x = T_y$



Name entity recognition

Recurrent Neural Networks的应用类型

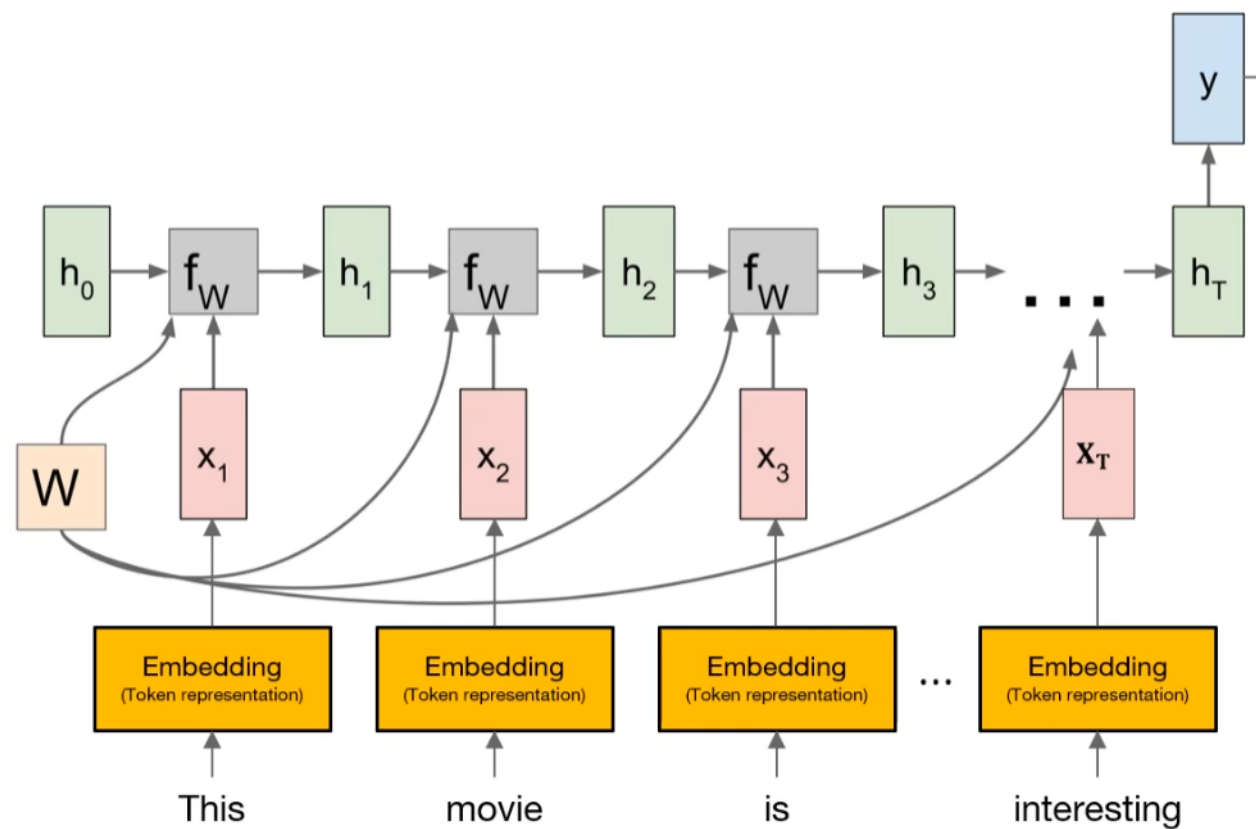
Many-to-many
 $T_x \neq T_y$



Machine translation

RNN 应用实例

- 情感分类例子: many to one
- Embedding: 自学



深度学习编程工具

- [Caffe](#) (Berkeley Vision and Learning Center)
- [CNTK](#) (Microsoft)
- [Deeplearning4j](#) (Skymind)
- [TensorFlow](#) (Google)
- [Theano](#) (University of Montreal + broad)
- [Torch](#) (Ronan Collobert)
- [Many others](#)



小结

- Convolutional Neural Network, CNN
 - Recurrent Neural Networks, RNN
- 深度学习的应用

References

- R. Salakhutdinov and G. Hinton. Deep Boltzmann Machines. ICAIS, 2009
- R. Salakhutdinov and G. Hinton. A Better Way to Pretrain Deep Boltzmann Machines. NIPS, 2013
- Jake Bouvrie(2014). Notes on Convolutional Neural Networks.
- Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-Based Learning Applied to Document Recognition, Proceedings of IEEE, Vol. 86, No. 11, pp. 2278-2324, 1998.
- Y. LeCun, Y. Bengio: Convolutional Networks for Images, Speech, and Time-Series, In Arbib, M.A. The hand book of Brain Theory and Neural Networks, MIT Press, 1995, 255-258
- S. Hochreiter and J. Schmidhuber. Long short-term memory. Neural Computation, 1997
- Ilya Sutskever, Oriol Vinyals, Quoc V. Le, Sequence to Sequence Learning with Neural Networks
- James Bergstra and Yoshua Bengio, Random Search for Hyper-Parameter Optimization (2012), in: Journal of Machine Learning Research, 13(281--305)
- ImageNet Classification with Deep Convolutional Neural Networks, Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton, NIPS 2012.
- Szegedy C, Liu W, Jia Y, et al. Going deeper with convolutions[J]. arXiv preprint arXiv:1409.4842, 2014.

References

- links to code

- sparse coding

- <http://www.eecs.umich.edu/~honglak/software/nips06-sparsecoding.htm>

- DBN

- <http://www.cs.toronto.edu/~hinton/MatlabForSciencePaper.html>

- DBM

- <http://web.mit.edu/~rsalakhu/www/DBM.html>

- convnet

- <http://cs.nyu.edu/~koray/publis/code/eblearn.tar.gz>

- <http://cs.nyu.edu/~koray/publis/code/randomc101.tar.gz>

- link to general website on deeplearning

- <http://deeplearning.net>

Thanks !